ABC80 inifrån

Utanför boken

Försök med det som kunde ha varit:
tolkar, språk och kort som aldrig fanns

Set Lonnert

Arbetsutkast

ABC80 inifrån. Utanför boken

© Set Lonnert 2026

Arbetsutkast. Häftet hör till *ABC80 inifrån*, band 1, Maskinen och BASIC:en, och band 2, Utbyggnaderna, och är fritt att läsa och sprida.

Häftet är skrivet av Set Lonnert tillsammans med Claude, en AI-modell från Anthropic, använd i verktyget Claude Code. Claude har skrivit och prövat programmen i emulatorn, mätt dem och formulerat texten utifrån författarens underlag. Set Lonnert har valt försöken, styrt arbetet och läst, rättat och godkänt texten. Ansvaret för innehållet är författarens.

Programmen och mätskripten finns i bokens kodförråd. Frågor och rättelser till `setlonnert@gmail.com`.

Innehåll

1 Inledning **1**
- 1.1 Försöken . . . 1
- 1.2 Hur försöken är gjorda . . . 2
- 1.3 Koden . . . 2

2 Två tolkar, samma förbättringar? **3**
- 2.1 Proven och mätningen . . . 3
- 2.2 Var tiden går . . . 4
- 2.3 Ändringarna . . . 5
  - 2.3.1 Satsslingan . . . 5
  - 2.3.2 Uttryckskoderna . . . 6
  - 2.3.3 Variabeln och värdet . . . 6
  - 2.3.4 Flyttalen . . . 7
  - 2.3.5 BASIC II . . . 7
- 2.4 Tiderna . . . 8
- 2.5 Att ändringarna håller . . . 9
- 2.6 Hade en omskrivning lönat sig? . . . 10
- 2.7 Vad som kom fram på vägen . . . 10
- 2.8 Inte prövat . . . 11

3 En snabbare Forth **12**
- 3.1 Tolken . . . 12
- 3.2 Var tiden går . . . 13
- 3.3 Ändringen . . . 14
- 3.4 Mätningen . . . 15
- 3.5 Vad det kostar . . . 15
- 3.6 Vad mer hade gått att vinna . . . 16
- 3.7 Inte prövat . . . 16

4 COMAL 0,5 **17**
- 4.1 COMAL och Compis . . . 17
- 4.2 Satserna . . . 18
- 4.3 Hur det är byggt . . . 19
- 4.4 Tiderna . . . 20
  - 4.4.1 ENDWHILE räknar villkoret . . . 20
  - 4.4.2 Platsen först . . . 21
  - 4.4.3 CASE utan kopior . . . 21
- 4.5 Gamla program . . . 22
- 4.6 Vad som kom fram på vägen . . . 22
- 4.7 Inte prövat . . . 22

5 Forthaid **23**
- 5.1 Så används den . . . 23
- 5.2 Hur den tar över . . . 24
- 5.3 Stegningen . . . 25
- 5.4 Minnet . . . 26
- 5.5 Tiderna . . . 27
  - 5.5.1 FIND . . . 27

5.5.2 VAR . . . 27
5.6 Orden i CMOS-minnet . . . 28
5.7 Begränsningarna . . . 28
5.8 Slutsatserna . . . 29
5.9 Inte prövat . . . 29

6 Forthaid II **30**
6.1 Varför Forthaid krockar med TKN80 . . . 30
6.2 Utan banker . . . 30
6.3 Bankkortet . . . 31
6.4 Fjärrorden . . . 32
6.4.1 Vad ett anrop kostar . . . 33
6.4.2 Korskompilatorn . . . 34
6.5 Kommandona i bankerna . . . 34
6.6 80 kolumner . . . 34
6.7 Tiderna . . . 35
6.8 Kompilatorn i en egen bank . . . 35
6.9 Orden på skivan . . . 36
6.10 Storleken . . . 37
6.11 Begränsningarna . . . 37
6.12 Slutsatserna . . . 37
6.13 Inte prövat . . . 38

7 Ett färggrafikkort i HR-80:s anda **39**
7.1 Annonsen . . . 39
7.2 Det som är gissat . . . 40
7.3 Tolken . . . 40
7.4 På V24 . . . 41
7.5 Exemplet . . . 42
7.6 Kortet i emulatorn . . . 43
7.7 Ett riktigt kort . . . 43
7.8 Begränsningarna . . . 43
7.9 Slutsatserna . . . 43
7.10 Inte prövat . . . 44

8 Fönster på ABC80 **45**
8.1 Demot . . . 45
8.2 Koderna . . . 45
8.3 JSP . . . 46
8.4 Fönstren på kortet . . . 47
8.5 Musen . . . 47
8.6 Händelserna . . . 48
8.7 Exemplet . . . 48
8.8 I emulatorn . . . 49
8.9 Storleken . . . 50
8.10 Begränsningarna . . . 50
8.11 Slutsatserna . . . 50
8.12 Inte prövat . . . 51

A Forth på ABC80 **52**
A.1 Stacken . . . 52
A.2 Egna ord . . . 53
A.3 BASIC-raderna . . . 54

B COMAL 0,5 på ABC80 **56**
B.1 Det nya 56
B.2 Ett program 56

Inledning

De två banden av *ABC80 inifrån* beskriver maskinen och programmen som de var: ROM:arna, kretsarna och korten som Luxor och andra byggde, lästa ur koden och prövade i en emulator. Banden håller sig till det som koden visar. Under arbetet kom ändå frågor som koden inte kan svara på, eftersom de gäller det som aldrig blev av. Hade ABC80-BASIC:en kunnat gå fortare utan att bli en annan? Vad hade COMAL:s strukturer kostat i den gamla tolken? Hur hade en verktygskrets som Smartaid sett ut om den hade varit öppen för användaren?

Det här häftet samlar försöken att svara. Vart och ett bygger på bandens kod och har prövats i samma emulator, men inget av det fanns på riktigt. Ju längre ett försök går från maskinen som den var, desto mindre säger det om ABC80 och desto mer om vad som hade varit möjligt med den. Därför börjar varje kapitel med vad som är ändrat och mot vad, och därför står försöken här och inte i banden.

1.1 Försöken

Kapitlen går från det som ligger närmast ABC80 som den var till det som ligger längst ifrån.

- *Två tolkar, samma förbättringar?* mäter ABC80-BASIC:en och BASIC II för ABC80 med samma prov, fördelar tiden på ROM:arnas rutiner och prövar ändringar i lappade kopior. Språket och talen är desamma; det som ändras är hur tolken tar sig fram till dem. ABC80-BASIC:en hade kunnat gå 14–31 % fortare så, med en tilläggskrets på `$5000`.
- *En snabbare Forth* tar Forth-tolken från band 2, kapitlet Egna exempel, och gör dess stackprövning billigare. Kapitlet resonerar också mer allmänt om vad som kostar tid i en trådad tolk och vad som går att vinna.
- *COMAL 0,5* ger ABC80-BASIC:en COMAL:s styrstrukturer, WHILE, REPEAT, block-IF, procedurer med lokala variabler och CASE, i 2 215 byte på `$4000` och 23 byte i ROM:en, och mäter dem mot GOTO, GOSUB och BASIC II.
- *Forthaid* är en verktygskrets i Smartaids stil, skriven i Forth på samma tolk och byggd för `$4000`–`$5FFF`, med skärmeditor och stegning. Kommandona är Forth-ord, och användaren kan skriva egna.
- *Forthaid II* flyttar Forthaid ur vägen för TKN80, kortet för 80 tecken per rad, och ger den mer plats genom bankning i den trådade tolken.
- *Ett färggrafikkort i HR-80:s anda* utgår från Libers annons för grafikkortet HR-80 (1984). Kortets manual och program har inte gått att hitta, så kapitlet bygger en egen tolk som gör det annonsen beskriver och tar emot kommandona på V24.

Två bilagor, Forth på ABC80 och COMAL 0,5 på ABC80, visar hur språken används, för den som vill pröva Forthaid och COMAL på emulatorns webbsida.

1.2 Hur försöken är gjorda

Bandens filer och ROM-avbildningarna ändras aldrig. Varje försök har ett skript som kopierar de källor det behöver, ändrar kopian och bygger av den. Det som ändras i en ROM ligger som lappar i kopian, och nya rutiner ligger, där ROM:en är full, i en krets på en ledig adress, som en tilläggs-ROM hade gjort.

Allt är prövat i emulatorn, inte på en riktig ABC80. Tiderna är emulatorns: de räknas i processorns T-cykler vid 2,9952 MHz eller med maskinens egen klocka, som räknar bildavbrotten 50 gånger i sekunden, och varje kapitel säger vilket. För BASIC II stämmer emulatorn med mätningar på en riktig ABC 802 inom ett par procent. Ett försök räknas som lyckat först när de gamla programmen och proven ger samma skärm, samma fel och samma tal som förut.

Kapitlen hänvisar till banden med kapitlets och avsnittets namn, utan sidnummer, eftersom häftet byggs för sig. Namn på rutiner och adresser är desamma som i banden.

1.3 Koden

Försöken finns i bokens kodförråd, bredvid bandens exempel och verktyg, med skripten som bygger och mäter dem och en beskrivning av vad som är ändrat:

```
https://github.com/Feyerabend/ABC80/tree/master/inifran/utanfor
```

Grafikkortet ligger hos emulatorn, i samma kodförråd. COMAL 0,5, Forthaid, Forthaid II och grafikkortet går också att pröva direkt i webbläsaren, i emulatorns webbsida:

```
https://feyerabend.github.io/ABC80/inifran/emulator/webb/abc80.html
```

Där väljs kretsen eller kortet i panelerna, eller med `?krets=comal`, `?krets=forthaid`, `?krets=forthaid2` och `?v24=hr80` efter adressen.

Två tolkar, samma förbättringar?

Utanför banden. Det som ändras är lappade kopior av ABC80-ROM:en (11273, den som band 1 beskriver) och av BASIC II för ABC80. Nya rutiner ligger i en krets på `$5000` för ABC80 och i oanvända byte i ROM:en för BASIC II. De mäts mot de olappade ROM:arna med samma 16 prov i emulatorn. Språket, koden som kompilatorn skriver och talen är desamma. Ingen av tolkarna fick någonsin de här ändringarna.

ABC80-BASIC:en skrevs 1978. BASIC II är ABC 800:s tolk, som omkring 1984–86 flyttades till ABC80, och band 2 beskriver den i kapitlen om BASIC II. Med samma program är BASIC II ofta nästan dubbelt så snabb. Frågan här är var den skillnaden sitter. Ligger den i tolkningen, alltså i hur tolken tar sig från en kod till nästa, eller i talen? Hade ABC80-BASIC:en kunnat gå fortare i en ny version, utan att bli ett annat språk? Hjälper samma ändringar båda tolkarna?

Försöket mäter tolkarna, fördelar tiden på ROM:arnas rutiner och prövar ändringar i kopior av ROM:arna. Tyngdpunkten ligger på ABC80.

2.1 Proven och mätningen

Tiden är emulatorns T-cykler vid 2,9952 MHz. Den räknas från att CR efter `RUN` trycks ned tills tolken är tillbaka i `READY`, alltså `$00CC` i ABC80 och `$0110` i BASIC II. Allt som skrivs på skärmen ingår. Mot en riktig ABC80 är tiderna inte prövade. För BASIC II stämmer de med mätningar på en riktig ABC 802: B3 tar 5,98 s i emulatorn och 5,91 s i maskinen, B7 19,83 mot 19,84 och MANDEL 92,6 mot 93.

Proven är 16 program ur kodförrådets mapp för ABC 802. B1–B7 är Rugg och Feldmans prov ur *Kilobaud* 1977, och B8 är det som *PCW* lade till 1978, här med 100 varv. NOEL och NOEL1 är oförändrade. De andra är ändrade så att ABC80 kan köra dem. `MOD(A,B)` är skrivet som `A-INT(A/B)*B`, `WHILE` som `IF` och `GOTO`, och tidtagningen är borttagen. Variabelnamnen har en bokstav och en siffra, och INTER går till 100 i stället för 1000. Utskriften är densamma i båda tolkarna. ABC80 har 16 kilobyte RAM och BASIC II 32.

Prov	ABC80	BASIC II	Kvot
B1	0,958	0,858	1,12
B2	2,115	1,814	1,17
B3	10,853	5,977	1,82
B4	11,827	5,998	1,97
B5	12,246	6,440	1,90
B6	17,485	11,683	1,50
B7	23,574	19,833	1,19
B8	13,318	2,886	4,61
FIZZ	3,573	2,229	1,60
FIZZ1	1,403	1,473	0,95
INTER	12,929	6,728	1,92
MANDEL	169,131	92,573	1,83
NOEL	24,418	18,766	1,30
NOEL1	9,333	7,839	1,19
PRIMES	93,562	44,262	2,11
SIEVE	26,312	30,655	0,86

Tiderna i tabellen är sekunder, och kvoten är ABC80:s tid delad med BASIC II:s. Den nya ABC80-ROM:en, 9913, går lika fort som den gamla i alla prov: bara PRIMES skiljer, med två tusendels sekund. Ändringarna nedan gäller därför den gamla.

Skillnaden är störst med flyttal. B8 räknar med `^`, `LOG` och `SIN` och går 4,6 gånger fortare i BASIC II, eftersom ABC80 räknar med BCD och BASIC II binärt. Med heltal är tolkarna nästan lika snabba, och med fält är ABC80 snabbare (SIEVE).

2.2 Var tiden går

Emulatorn kan räkna T-cykler för varje adress i ROM:en. Summerade per rutin, med namnen ur listningarna, visar de var tiden går.

- *Flyttalen.* I B4 går 27 % av ABC80:s tid i BCD-divisionen, slingan `FDLOOP`, och omkring 30 % till i multiplikationen och skiftningarna. I BASIC II går 17 % till `FDIV` och 13 % till `FMUL`. ABC80:s multiplikation hoppar redan över nollor i multiplikatorn, men bygger en tabell med multiplikanden gånger 2, 4 och så vidare för varje multiplikation. Band 1 beskriver båda rutinerna i kapitlet Flyttalen.
- *Uttryckskoderna.* ABC80 går genom `EXEXP` för varje kod i uttrycket: `RST $28`, två `INC SP`, ett hopp, `PUSH HL` och uppslaget i tabellen, omkring 135 T per kod. Det är 17–25 % av tiden i proven med heltal. BASIC II lägger lika stor del på motsvarande rutin, `EXECA`, men där gör varje kod mer, och koderna kedjas med `RST $28` till `EXECNX` för omkring 98 T.
- *Raderna och satserna.* ABC80 prövar TRACE för varje rad och anropar `TSTBRK` för varje sats, 82 T. `RUNLIN` och `RUNSTM` tar tillsammans 20 % av tiden i NOEL1. BASIC II prövar BREAK, TRACE och CTRL-S på en gång med en flaggbyte, på 35 T.
- *Variablerna.* Ett variabelvärde är två koder i ABC80: `VAR0` med variabelns adress och sedan `GETF` eller `GETI`, som byter adressen mot värdet. I BASIC II är det en kod, `LDFVAR` eller `LDIVAR`.
- *Fälten.* BASIC II multiplicerar indexet med elementets storlek i `MUL16`, som alltid går 16 varv och tar omkring 1000 T. Det är 17 % av SIEVE och skälet till att ABC80 är snabbare där.
- *FOR och NEXT.* `NEXT` är 20 % av NOEL1 i BASIC II och 9 % i ABC80.

Tiden sitter alltså i två slags ställen. Det ena är tolkningen: satsslingan, uttryckskoderna och variablerna, där varje varv kostar lite men görs många gånger. Det andra är talen, där varje multiplikation och division kostar mycket.

2.3 Ändringarna

Varje ändring är en lapp i en egen fil. Ett skript bygger en lappad kopia av ROM:en, och för ABC80 också kretsen på `$5000`. Alla ändringar följer samma villkor: inget i språket ändras, och koden som kompilatorn skriver är densamma, så att sparade program går att köra som förut. Talen räknas med samma BCD och samma avrundning. Det som ändras är bara hur ROM:en tar sig fram till samma resultat, med medel som fanns 1978: HL i stället för `(IX+n)`, tabeller i stället för slingor, en prövning mindre per sats och ett mönster som tolken känner igen.

Lapp	Tolk	Byte	Vad den gör
`a_brk`	ABC80	8	BREAK-flaggan läses direkt i `RUNSTM`
`a_tab`	ABC80	19+512	`EXEXP` slår upp koden i två tabeller på hela sidor
`a_var`	ABC80	35	`VAR0` hämtar värdet direkt när nästa kod är `GETF` eller `GETI`
`a_sup`	ABC80	297	som `a_var`, och `VAR0` känner igen `V=V+X` och `V=V-X` och gör hela satsen
`a_madd`	ABC80	37	additionen i `FMUL` med HL i stället för `(IX+n)`
`a_fswp`	ABC80	39	`FMUL` byter faktorerna så att den med flest nollor i låga änden blir multiplikator
`a_ftab`	ABC80	262	`FMUL`:s tabell byggd utan slingor, med HL och DE
`a_fdiv`	ABC80	72	subtraktionerna i `FDLOOP` med HL och DE i stället för `(IX+n)`
`b_mul`	BASIC II	28	indexet gånger storleken med en slinga som slutar när storleken är slut, 2–3 varv
`b_nxt`	BASIC II	95	`NEXT` för en heltalsvariabel med HL i stället för `(IX+n)`
`b_sup`	BASIC II	260	satsslingan känner igen `V=V+X` och `V=V-X` och gör satsen själv

Lapparna mäts en och en och i fyra uppsättningar. `a_hel` är `a_brk`, `a_tab` och `a_sup`, alltså tolkningen i ABC80. `a_flt` är de fyra flyttalslapparna. `a_alla` är båda, och `b_alla` är de tre lapparna i BASIC II. `a_var` ingår i `a_sup`.

ABC80:s ROM är nästan full. De nya rutinerna ligger därför i en krets på `$5000`, som en tilläggs-ROM, och ROM:en får bara de hopp som leder dit. I BASIC II ligger lapparna på `$0450`–`$05D3`. Där står bara nollor (`$044A`–`$06A9`), eftersom ABC 800:s drivrutin för kassetten, `CAS:`, togs bort i ABC80-versionen (band 2, kapitlet BASIC II på ABC80 och ABC 800).

2.3.1 Satsslingan

Band 1 beskriver `RUNSTM` i kapitlet Kommandona, avsnittet Körningen. Före varje sats byter den DE och HL, anropar `TSTBRK` och byter tillbaka. `TSTBRK` läser BREAK-flaggan, prövar den och nollställer den, varje gång:

```
0D92  EB           RUNSTM: EX   DE,HL
0D93  CD 3E 03             CALL TSTBRK          BREAK-flaggan, nollställd
0D96  C2 23 23             JP   NZ,BRKSTP       BREAK: STOP
0D99  EB                   EX   DE,HL
0D9A  1A                   LD   A,(DE)          satsens första byte
0D9B  FE 0D                CP   $0D
0D9D  C8                   RET  Z               CR: raden är slut
0D9E  CD A3 0D             CALL EXESTM          satsen
0DA1  18 EF                JR   RUNSTM
```

De fyra första instruktionerna tar 82 T. Lappen `a_brk` skriver i stället samma åtta byte så här:

```
0D92  3A 07 FE     RUNSTM: LD   A,(BRKFLG)      flaggan läses på plats
0D95  A7                   AND  A
0D96  C2 93 50             JP   NZ,BRKSTB       BREAK: till kretsen
0D99  00                   NOP                  fyllnad
```

Det blir 27 T och en `NOP`. Flaggan nollställs bara när BREAK verkligen har kommit, i en kort rutin i kretsen:

```
5093  EB           BRKSTB: EX   DE,HL           som efter TSTBRK
5094  AF                   XOR  A
5095  32 07 FE             LD   (BRKFLG),A      nollställd, som i TSTBRK
5098  C3 23 23             JP   BRKSTP
```

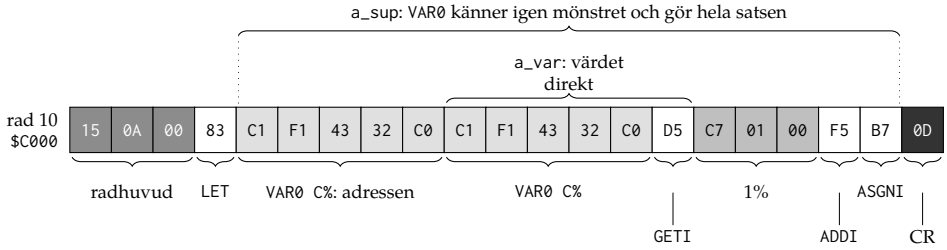
Båda instruktionerna i `BRKSTB` är egentligen onödiga. `BRKSTP` ger fel 0, och felrutinen skriver över HL och DE. Fel 0 leder alltid till `READY`, som anropar `TSTBRK` och nollställer flaggan där. De står kvar för att rutinen ska göra precis det som ROM:en gjorde. Lappen ger 3,5–7,3 % för åtta byte i ROM:en och nio i kretsen.

2.3.2 Uttryckskoderna

`EXEXP` (band 1, kapitlet Uttrycken, avsnittet Körningen av uttryckskoden) slår upp varje kod i tabellen `EXPTAB1` och sätter B, C och CY innan den hoppar till rutinen. Lappen `a_tab` lägger rutinernas adresser i två tabeller på hela sidor, `$5E00` för de låga byten och `$5F00` för de höga, så att koden själv blir indexet. Den första versionen tog 54 T i stället för 91 men satte inte B och C som ROM:en, och den gav fel i alla prov med funktioner. Med samma register som ROM:en återstår 82 T, nio färre per kod. Det ger 0,5–2,4 % för 512 byte tabeller, och av lapparna är det den sämsta affären räknat i byte.

2.3.3 Variabeln och värdet

Figuren visar hur ABC80-ROM:en kompilerar satsen `C%=C%+1%` (band 1, kapitlet Operatorerna, avsnittet Variablerna vid körning). Variabeln till vänster blir `VAR0` med adressen, eftersom det är dit värdet ska. Variabeln till höger blir `VAR0` med samma adress och sedan `GETI`, som byter adressen mot värdet. Konstanten blir `INTCON`, plus blir `ADDI` och tilldelningen `ASGNI`. Det är sex koder, och varje kod är ett varv genom `EXEXP`.



Raden `10 C%=C%+1%` *som ABC80-ROM:en lagrar den i minnet, här från* `$C000`*. Raden börjar med längden och radnumret, och CR (svart) avslutar den. Grå byte är variabelns kod med namnet* (`$F1`, `$43`) *och adressen* `$C032`*, och mörkgrå är konstanten 1. Under remsan står de koder som* `EXEXP` *går igenom en i taget. Över den står vad lapparna tar hand om i ett svep:* `a_var` *gör* `VAR0` *och* `GETI` *till ett varv, och* `a_sup` *gör hela satsen från den första* `VAR0`*.*

Lappen `a_var` ändrar `VAR0`:s tre poster i `EXPTAB1` så att de pekar på en ny rutin i kretsen, `VAR0X`. Den läser adressen som förut och ser sedan efter vilken kod som kommer härnäst. Är det `GETI` (`$D5`) eller `GETF` (`$D4`) hämtar den värdet direkt och hoppar över koden:

```
51A0  1A          LD   A,(DE)      nästa kod
51A1  FE D5       CP   $D5         GETI?
51A3  28 05       JR   Z,GETIX
51A5  FE D4       CP   $D4         GETF?
51A7  28 07       JR   Z,GETFX
51A9  EF          RST  $28         annars adressen, som förut
```

Koden som kompilatorn skriver är densamma, så sparade program går som förut. Lappen ger upp till 9,8 % för 35 byte.

Lappen `a_sup` går längre. När `VAR0` lägger adressen för den variabel som tilldelas, ser den efter om resten av satsen är just mönstret `V=V+X` eller `V=V-X`: samma adress med `GETI` eller `GETF`, en konstant eller en variabel av samma typ, plus eller minus, och tilldelningen. Stämmer det gör `VAR0` hela satsen själv. Heltal räknas med `ADD` och `SBC` som i `ADDI` och `SUBI`, och flyttal med ROM:ens `FADD` och `FSUB`. Variabeln skrivs först när räkningen har gått bra, så att fel 4 lämnar den orörd. Koderna läses i tur och ordning från början av en kod, så mönstret kan inte hittas mitt i en annan kods byte.

Mönstret är ingen ny kod, och `V=V+X` är en stor del av de inre slingorna. I NOEL1, där `S%=S%+J%` står i den inre slingan, ger lappen 23,0 % mot 9,8 med bara `a_var`. Det kostar 34 T i en tilldelning där nästa kod inte är en variabel och 104 T när den är en annan variabel av samma typ, som i `M=N/K` i INTER. Där blir vinsten 1,3 % i stället för 2,0.

2.3.4 Flyttalen

Flyttalslapparna skriver om delar av `FMUL` och `FDIV` (band 1, kapitlet Flyttalen, avsnitten Multiplikation och Division) utan att ändra algoritmen. Den största vinsten ger `a_fdiv`. Den gör subtraktionerna i divisionens inre slinga med HL och DE i stället för `(IX+n)`, där varje byte kostar 19 T att läsa och lika mycket att skriva. Den ger upp till 22 % för 72 byte (INTER 21,6, PRIMES 18,2). `a_madd` och `a_ftab` gör samma sak för multiplikationen, och tillsammans ger de B8 och MANDEL omkring 7–15 %.

`a_fswp` beror på programmet. När den ena faktorn är ett litet heltal eller har få siffror lönar det sig att byta, och B3 och MANDEL vinner omkring två procentenheter. Annars kostar prövningen 0,3–0,8 %. Att den är med i `a_alla` är ett val för just de här proven.

2.3.5 BASIC II

I BASIC II prövas tre ändringar. `b_mul` ersätter `MUL16` i fältindexet med en slinga som slutar när elementets storlek är slut, två eller tre varv i stället för 16. Den ger SIEVE 13,9 % och B7 6,4 %. `b_sup` är samma mönster `V=V+X` som `a_sup`. Satsslingan anropar `EXECA` för satsens första kod, och lappen tar just det anropet, så koderna inne i ett uttryck inte kostar något extra. Stämmer inte mönstret går lappen direkt till `LDIVAR` eller `LDFVAR`. Den ger NOEL1 16,7 %, mindre än i ABC80, eftersom BASIC II redan har en kod per variabel.

`b_nxt` ger nästan ingenting. Den gör `NEXT` för heltalsvariabler med HL genom FOR-blocket och lämnar flyttalen åt ROM:en. NOEL1 vinner 1,4 %, och sju prov förlorar lite, mest B1 med 0,6 %, eftersom hoppet till lappen kostar omkring 15 T per `NEXT` med flyttal. En första version gjorde om arbetet för flyttalen och förlorade 5,1 % i B1.

2.4 Tiderna

Tabellen visar vinsten i procent av tiden för lapparna i ABC80 och för uppsättningarna. Skärmen är densamma som utan lapparna i alla 16 prov och alla uppsättningar.

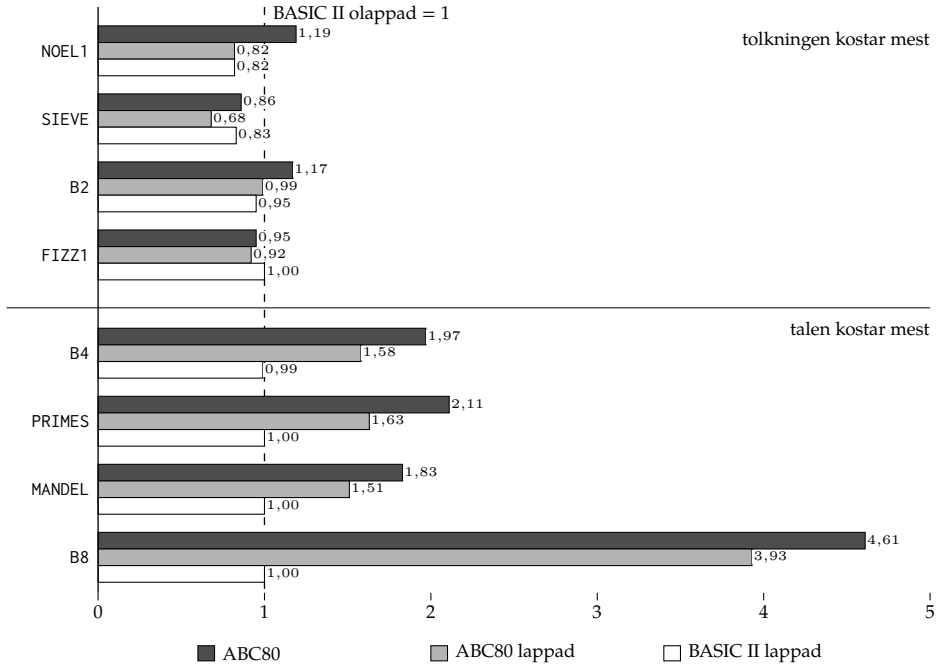
Prov	a_brk	a_var	a_sup	a_hel	a_fdiv	a_flt	a_alla	b_alla
B1	3,5	0,0	0,0	3,5	0,0	0,0	3,5	−0,6
B2	3,3	3,3	10,9	14,8	0,0	0,0	14,8	4,6
B3	0,9	2,3	3,4	4,8	7,5	14,0	18,8	1,4
B4	0,9	0,9	1,9	3,2	14,2	16,6	19,8	1,4
B5	1,3	0,8	1,9	3,5	13,7	16,0	19,5	1,3
B6	2,0	0,6	1,2	3,6	9,6	11,2	14,8	0,5
B7	2,2	2,0	2,4	5,3	7,1	8,3	13,6	6,6
B8	0,1	0,1	0,2	0,3	0,0	14,5	14,8	0,1
FIZZ	0,5	0,6	0,6	1,3	15,1	17,1	18,4	0,0
FIZZ1	1,3	1,9	1,9	3,8	0,0	0,0	3,8	0,1
INTER	2,0	2,0	1,3	3,9	21,6	21,6	25,5	−0,1
MANDEL	1,5	2,7	2,7	4,7	0,0	12,7	17,5	0,0
NOEL	2,8	2,9	10,9	14,1	0,0	0,0	14,1	5,6
NOEL1	7,3	9,8	23,0	31,2	0,0	0,0	31,2	18,1
PRIMES	0,8	1,3	1,3	2,5	18,2	20,5	23,0	0,0
SIEVE	6,4	9,0	12,7	20,6	0,0	0,0	20,6	17,2

Med alla lapparna går ABC80-BASIC:en 14–31 % fortare i de flesta proven. B1 och FIZZ1 vinner bara 3,5–3,8 %, eftersom de mest består av en tom slinga och utskrift. Vinsterna läggs ungefär ihop, eftersom lapparna rör olika rutiner. `a_alla` är omkring 730 byte kod och 512 byte tabeller i kretsen, plus några hopp i ROM:en.

Den intressanta jämförelsen är när båda tolkarna är lappade. Figuren visar tiden för åtta av proven, räknad i den olappade BASIC II:s tid.

Där tolkningen är det som kostar blir ABC80 lika snabb som BASIC II, också när BASIC II lappas på samma sätt. NOEL1 tar 6,42 s i båda, B2 1,80 mot 1,73 och SIEVE 20,9 mot 25,4. BASIC II:s tolk är alltså inte i grunden snabbare. Där talen är det som kostar ligger BASIC II kvar långt före: B8 tar 11,3 s mot 2,9 och MANDEL 139,6 mot 92,6. Skillnaden är BCD mot binära flyttal, och den rår ingen ändring i tolkningen på.

Samma grepp hjälper båda tolkarna, men olika mycket. Mönstret `V=V+X` ger 23 % i ABC80 och 17 % i BASIC II (NOEL1). Slingan i fältindexet ger 14 % i BASIC II (SIEVE), och ABC80 behöver den inte. `NEXT` med HL i stället för IX ger nästan ingenting.



Tiden för åtta prov som andel av den olappade BASIC II:s tid (den streckade linjen vid 1). Den mörka stapeln är ABC80, den grå ABC80 med `a_alla` *och den vita BASIC II med* `b_alla`*. I de fyra översta proven är det tolkningen som kostar, och där når den lappade ABC80 BASIC II. I de fyra nedersta är det talen, och där ligger BASIC II kvar långt före.*

2.5 Att ändringarna håller

Att proven ger samma skärm visar bara att ändringarna fungerar för det som de 16 proven gör. De är därför prövade på fyra sätt till.

– *Rutinerna direkt.* Ett provprogram anropar `FMUL` och `FDIV` i den olappade och den lappade ROM:en med samma tal och jämför resultatets fem byte, flaggorna, registren och fel 4. Talen är kanttal, som de största och minsta exponenterna och mantissor som 999999 och 500005, tal med en till tre siffror och slumpade tal. Det blir 300 000 fall per rutin och uppsättning, utan skillnader. `b_mul` är prövad på samma sätt mot `MUL16` i 2,7 miljoner fall.
– *Bandens tolkprov.* De 569 proven för tolkarna i kodförrådet körs med de lappade ROM:arna. 545 ger samma skärm. De 24 andra skiljer bara i det som beror på tiden: de stannar efter ett visst antal instruktioner, tar slumptalets frö ur R-registret eller trycker BREAK efter en viss tid.
– *Kantfallen.* 63 små program körs i båda tolkarna, olappade och lappade. De täcker FOR och NEXT med alla slags steg och gränser, fält med index utanför, flyttal nära spill och heltal vid ±32767. Där finns också satser som nästan är mönstret, som `A%=A%+1%*2%` och `A%=B%+1%`, och spill mitt i satsen. Alla 126 körningar ger samma skärm.
– *Medvetna fel.* Lapparna byggs med fel inlagda för att se att proven märker dem. I `a_sup` märks 14 av 20 fel och i `b_sup` 18 av 25. Vart och ett av de andra är förklarat ur listningen: prövningar som aldrig kan slå till, eftersom kompilatorn alltid skriver `GETI` eller `GETF` efter en variabel i ett uttryck, eller register som ingen läser efter satsen.

Det sista provet säger mest. Ett fel som inget prov märker är antingen ofarligt eller ett hål i proven, och när ett nytt hål hittades lades kantfall till. Subtraktion av två heltal fanns till exempel först inte bland kantfallen, eftersom konstanten i `A%-1` är ett flyttal och ABC80 då räknar med flyttal.

2.6 Hade en omskrivning lönat sig?

En omskrivning av ABC80-BASIC:en finns redan att titta på. BASIC II har binära flyttal, `DOUBLE` och en annan kod, och den behöver 30 kilobyte ROM i stället för 16. Mätningen delar upp skillnaden mellan tolkarna i två delar. I tolkningen når den lappade ABC80-BASIC:en BASIC II, och en ny tolk hade inte gett mycket mer. Det som återstår är kostnaden när mönstret inte stämmer och de 135 T per uttryckskod i `EXEXP`, som bara en ny kod kunde ta bort. I talen ligger BASIC II kvar långt före, och det är bara en ny talform som tar bort den skillnaden.

En omskrivning hade kostat mer än platsen i ROM:en.

- *Programmen.* `SAVE` skriver den kompilerade koden, och med en ny kod går ingen sparad BAC-fil att läsa. Program sparade som text går att läsa in och kompilera om.
- *Tilläggen.* DOS-ROM:arna anropar ROM:ens hoppvektorer och tre inre rutiner, och Superbasic och Schoolaid anropar 44 adresser var inne i ROM:en. En ny tolk måste ha kvar varje sådan ingång, med samma register, annars måste tilläggen skrivas om. Lapparna lämnar ingångarna där de var.
- *Talen.* BCD räknar 0,1 exakt, och binära flyttal gör det inte. En ABC80 med binära flyttal hade skrivit andra siffror i en del program, och `FOR` med `STEP 0.1` eller `IF X=0.3` hade kunnat ta en annan väg. Det är en ändring av språket, inte av tolken.
- *Utrymmet.* Redan lapparna behöver en krets till. 1978 kostade varje ROM-krets pengar i varje maskin.
- *Att pröva den.* Varje lapp här har ett facit: den olappade ROM:en ska ge exakt samma svar. En ny tolk har inget facit utom manualen och programmen.

Om ABC80-BASIC:en hade fått en ny version, hade det lönat sig att göra som här, men från början och i ROM:en: behålla ingångarna, BAC-koden och BCD-formatet, och ta `a_brk`, `a_sup`, som har `a_var` i sig, och `a_fdiv`. Det är 377 byte för det mesta av vinsten. Tabellerna i `a_tab` och bytet i `a_fswp` hade fått stå tillbaka. Det enda som en hel omskrivning hade köpt utöver det är flyttalen. Det priset betalade BASIC II: en annan kod, andra tal, ett nytt gränssnitt mot DOS och dubbelt så mycket ROM. För en maskin som redan hade sina program, tillägg och användare var det en ny maskin, inte en ny version.

2.7 Vad som kom fram på vägen

Försöket hittade några saker i ROM:arna som inte syns förrän något ändras.

- *DIM-felet i ABC80:s ROM.* När `DIM` går fel i `CMPDMI`, till exempel med `DIM A` utan index, ligger ett registerpar kvar på stacken, och felrutinen hoppar till en adress som beror på variabelns namn: bokstaven blir hög byte. Band 1 beskriver felet i bilagan Fel och egenheter i tolken. På en vanlig ABC80 står `$FF` där, `RST $38`, och det blir fel 4 av en slump. Kretsen låg först på `$4000`, och då hoppade felet in i tabellerna. Nu ligger den på `$5000` med lapparna i luckorna `$x93`–`$xEF`, dit felet inte kan hoppa, och byggskriptet vägrar lägga kod någon annanstans.
- `$7800`–`$7FFF` *är inte ledigt i BASIC II.* Området är `$FF` i ROM-filen, och lapparna låg där först. Men i maskinen är `$7800`–`$7BFF` RAM för skivminnets DOS och `$7C00` och uppåt skärmen. Bandens prov skrev sina filbuffertar över lapparna.
- *BREAK som tappas.* `TSTBRK` läser flaggan, prövar den och skriver 0. Kommer tangentavbrottet mellan läsningen och skrivningen försvinner BREAK. `CHKBRK` i IEC-ROM:en gör likadant. Lappen `a_brk` flyttar bara var det kan hända.
- *Heltal som slår runt.* I båda tolkarna blir `FOR I%=1 TO 32767` aldrig klar, eftersom `I%` går från 32767 till −32768. ABC80 märker inte heller spill när heltal multipliceras: 182·182 blir −32412.

– *BASIC II:s referenser.* En variabels referens i BASIC II:s kod är inte ett löpnummer utan beror på namnet. A1 och Q1 har samma låga byte, och det är så ett av kantfallen prövar att lappen jämför båda byten.

2.8 Inte prövat

Flyttalen i BASIC II är inte lappade. De är redan binära och skrivna med register, och tyngdpunkten här är ABC80. En annan algoritm för BCD-divisionen är inte prövad heller, eftersom lapparna bara skriver om de gamla rutinerna med andra register och tabeller. Tiderna är emulatorns, och bara BASIC II är jämförd med en riktig maskin.

Lapparna, skripten och proven ligger i `utanfor/tolkjamforelse` i kodförrådet. Ett skript, `allt.sh`, gör om allt, från proven till de medvetna felen.

En snabbare Forth

Utanför banden. Det som ändras är en kopia av Forth-tolken från band 2, kapitlet Egna exempel, avsnittet Exempel 7: en liten Forth, byggd som krets på `$4000`. Bara ordet `BRANCH` är utbytt, och allt annat är som i bandet. Varianten mäts mot bandets tolk och mot ABC80-BASIC:en med samma prov i emulatorn. Tolken fanns aldrig på riktigt, och inte heller varianten.

Forth-tolken i band 2 är försiktig. Den körs från BASIC med `CALL`, och dess stackar ligger direkt under BASIC:s stack, så ett fel i ett Forth-program kan skriva över BASIC-programmet och dess variabler. Därför prövar tolken båda stackarna i varje varv av varje slinga och vid varje anrop av ett ord. Frågan här är vad den försiktigheten kostar och vad som går att vinna genom att pröva mindre. Bilagan Forth på ABC80 visar hur ett Forth-program skrivs, för den som inte har gjort det förut.

En trådad tolk är lätt att räkna på. Varje ord är några få instruktioner, och vägen genom dem är nästan alltid densamma. Tiderna nedan är därför räknade på två sätt: i T-cykler ur listningen och med maskinens klocka i emulatorn. De stämmer med varandra inom en procent. Det gör det också möjligt att resonera om ändringar som inte är prövade.

3.1 Tolken

Tolken är direkt trådad. Ett ord i maskinkod slutar med ett hopp till `NEXT`, som hämtar nästa adress i tråden och hoppar dit, och ett ord i Forth börjar med `CALL DOCOL` följt av sin tråd. Datastacken är Z80:s egen stack, och det senast lagda värdet ligger i BC. Returstacken pekas ut av IX och växer nedåt, DE är nästa adress i tråden och HL är arbetsregister. Band 2 beskriver tolken i avsnitten Trådad kod och Stacken kontrolleras.

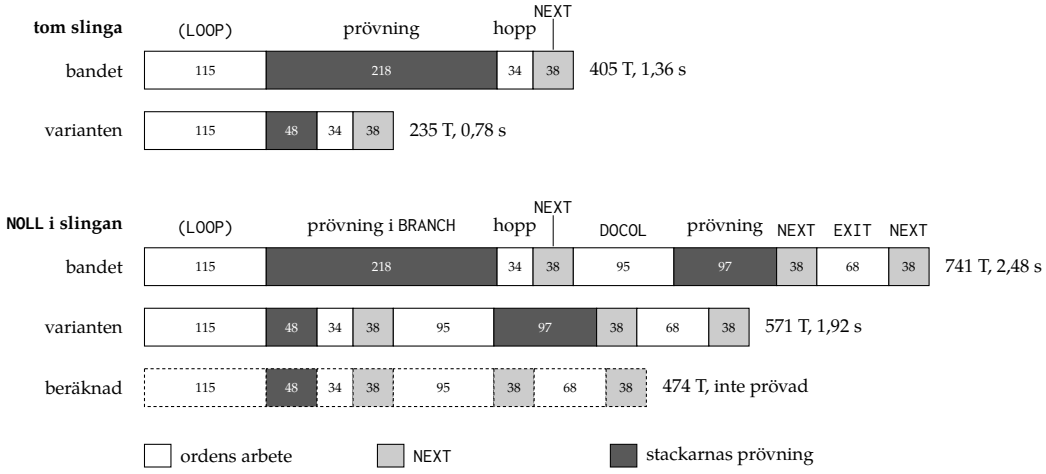
`NEXT` är sju byte och 38 T, och hoppet dit från ordets slut är 10 T till. Ett ord som `+`, som självt tar 29 T, kostar alltså 77 T med hoppet och `NEXT`. Det är priset för tolkningen, och det betalas en gång per ord. Allt som tolken gör utöver orden ligger i `NEXT`, i `DOCOL` och `EXIT` vid anrop och i `BRANCH` vid hopp.

`BRANCH` hoppar till adressen som står efter den i tråden. Alla slingor går igenom den: `(LOOP)` när slingan ska gå ett varv till, `AGAIN` och `REPEAT` alltid och `0BRANCH` när den hoppar. I bandets tolk är det här stackarna prövas mitt i ett ord. Tre saker prövas: att datastacken inte har tömts förbi sin botten, som står på `$7C7A` (`S0`), att den inte har vuxit ned till 64 byte från BASIC:s variabler, vars slut står på `$FE20` (`HEAP`), och att returstacken inte har vuxit ned till kanariefågeln, värdet `$5AA5` mellan de två stackarna. `DOCOL` prövar returstacken på samma sätt vid varje anrop, och efter varje rad prövar `?STACK` att datastacken inte har tömts och att kanariefågeln står kvar.

3.2 Var tiden går

Den tomma slingan `10000 0 DO LOOP` kör `(LOOP)` och `BRANCH` varje varv. `(LOOP)` räknar upp indexet på returstacken och jämför det med gränsen, 115 T fram till hoppet till `BRANCH`. `BRANCH` prövar stackarna i 218 T, hämtar den nya adressen i 34 T, och `NEXT` tar 38 T. Det blir 405 T per varv, och 10 000 varv tar 1,35 s vid 2,9952 MHz. Klockan mäter 1,36 s. Av varvet går alltså mer än hälften till att pröva stackarna, och bara 115 T till det som slingan gör.

Ett anrop av ett tomt kolonord, `: NOLL ;`, kostar 336 T: `NEXT` till ordet, `CALL DOCOL` på 17 T, `DOCOL` på 175 T, `NEXT` till `EXIT` och `EXIT` med sitt hopp på 68 T. Av de 175 T i `DOCOL` är 97 prövningen av returstacken. Utan den hade `DOCOL` bara lagt DE på returstacken och tagit trådens adress från stacken, 78 T. En slinga som anropar `NOLL` 10 000 gånger blir 741 T per varv, 2,47 s, och klockan mäter 2,48 s. Figuren visar varven uppdelade.



T-cyklerna i ett varv av den tomma slingan och av slingan som anropar `NOLL`*, räknade ur listningarna, med klockans tid för 10 000 varv. Vitt är det som orden själva gör, ljusgrått* `NEXT` *och mörkt stackarnas prövning. Varianten byter bara prövningen i* `BRANCH`*. Den streckade raden är räknad men inte prövad: varianten med en* `DOCOL` *som inte prövar returstacken.*

Som jämförelse tar ABC80-BASIC:ens tomma slinga `FOR I%=1% TO 10000%` och `NEXT I%` 2,20 s, alltså omkring 660 T per varv. BASIC:en går också igenom en kod i taget, men varje kod slås upp i en tabell. Kapitlet Två tolkar, samma förbättringar? räknar det till omkring 135 T per kod i `EXEXP`, mot 48 T för `NEXT` med hoppet dit. Forth vinner inte på att orden gör mindre. Den vinner på att varje steg mellan dem är billigare, och den tappar mycket av det på prövningarna.

3.3 Ändringen

I bandet ser BRANCH ut så här. T-cyklerna för varje prövning står efter dess sista instruktion.

```
40AA  2A 7A 7C     BRANCH: LD   HL,(S0)          datastackens botten
40AD  B7                   OR   A
40AE  ED 72                SBC  HL,SP
40B0  DA 4D 40             JP   C,STKERR         tömd: SP över S0, 45 T
40B3  D5                   PUSH DE               trådens adress undan
40B4  21 C0 FF             LD   HL,-64
40B7  39                   ADD  HL,SP            SP − 64
40B8  ED 5B 20 FE          LD   DE,(HEAP)        BASIC:s variabler
40BC  B7                   OR   A
40BD  ED 52                SBC  HL,DE
40BF  DA 4D 40             JP   C,STKERR         nere vid dem, 81 T
40C2  DD E5                PUSH IX
40C4  D1                   POP  DE               DE = IX
40C5  2A 7A 7C             LD   HL,(S0)
40C8  23                   INC  HL
40C9  23                   INC  HL               över kanariefågeln
40CA  EB                   EX   DE,HL
40CB  ED 52                SBC  HL,DE            IX − (S0 + 2), C = 0 här
40CD  D1                   POP  DE               trådens adress tillbaka
40CE  DA 4D 40             JP   C,STKERR         returstacken, 92 T
40D1  EB                   EX   DE,HL            hoppet
40D2  5E                   LD   E,(HL)
40D3  23                   INC  HL
40D4  56                   LD   D,(HL)           DE = nya adressen
40D5  C3 26 40             JP   NEXT             34 T
```

Prövningarna behöver HL och DE, och eftersom DE är trådens adress måste den läggas undan och hämtas tillbaka. Hälften av tiden går åt till att flytta värden mellan registren, inte till att jämföra dem.

Varianten behåller bara den andra prövningen, den som skyddar BASIC-programmet, och gör den på den höga byten. Den slår till om SP − 64 ligger på samma sida om 256 byte som slutet på BASIC:s variabler eller lägre:

```
40AA  21 C0 FF     BRANCH: LD   HL,-64
40AD  39                   ADD  HL,SP            H = höga byten av SP − 64
40AE  3A 21 FE             LD   A,(HEAP+1)       höga byten av HEAP
40B1  BC                   CP   H
40B2  D2 4D 40             JP   NC,STKERR        samma sida eller lägre, 48 T
40B5  EB                   EX   DE,HL            hoppet som förut
40B6  5E                   LD   E,(HL)
40B7  23                   INC  HL
40B8  56                   LD   D,(HL)
40B9  C3 26 40             JP   NEXT             34 T
```

Prövningen kostar 48 T i stället för 218 och behöver varken DE eller något RAM. Den slår till upp till 256 byte för tidigt, men mellan datastacken och BASIC:s variabler finns för det mesta flera kilobyte. De två andra fallen, en tömd datastack och en returstack som vuxit för långt, fångar ?STACK efter raden, som i bandet, och DOCOL prövar fortfarande returstacken vid varje anrop. Varianten är 28 byte kortare än bandets tolk. Ett skript kopierar bandets källor, byter ut BRANCH i kopian och bygger kretsen på samma sätt som bandets.

3.4 Mätningen

Proven körs i BASIC och i båda tolkarna, med 10 000 varv utom i sållet, som söker primtalen upp till 1000 och finner 168 i alla tre. ROM:en är den nya ABC80-ROM:en. Tiden mäts med maskinens egen klocka på 65008, som räknas ned 50 gånger i sekunden, och upplösningen är därför 20 ms. Klockans låga byte räknar från 256 ned till 1, och 0 betyder 256, som band 1 beskriver i kapitlet ROM:en och hårdvaran, avsnittet Klockan (NMI). Både BASIC-programmet och Forth-orden som mäter lägger därför till 256 när den låga byten är 0; utan det blev en av mätningarna 5,12 s fel.

Prov	BASIC	Forth, bandet	Forth, varianten
tom slinga	2,20	1,36 (1,6)	0,78 (2,8)
tom slinga, BASIC med flyttal	9,26	1,36 (6,8)	0,78 (11,9)
`A%=I%+I%-5%`, `I I + 5 - A !`	8,94	3,50 (2,6)	2,92 (3,1)
`GOSUB`, anrop av `NOLL`	5,60	2,48 (2,3)	1,92 (2,9)
sållet till 1000	2,82	1,72 (1,6)	1,48 (1,9)
`BEGIN 1 - DUP 0= UNTIL`	–	2,08	1,50

Tiderna är sekunder, och inom parentes står hur många gånger fortare Forth är än BASIC. Mätningen är körd två gånger, 2026-10-10 och 2026-10-27, med samma tider.

Den tomma slingan går nästan dubbelt så fort, eftersom prövningen var mer än hälften av den. I proven där varje varv gör mer blir vinsten mindre, eftersom det som tar tiden där är orden, `NEXT` och `DOCOL`. I raden `I I + 5 - A !` kör varje varv sju ord, och de tar omkring 640 T med eller utan ändringen, ungefär 92 T per ord, varav 48 är `NEXT` med hoppet dit. Sållet är delat i korta ord, eftersom tolkens radbuffert tar 79 tecken, och tolken har varken `*` eller `/`, så proven räknar bara med `+` och `-`.

3.5 Vad det kostar

Varje fel prövas med en slinga som inte tar slut, och sedan med `1 2 + .` och `BYE`, för att se om tolken och BASIC finns kvar.

Slinga	Bandet	Varianten
`BEGIN 1 AGAIN`, fyller datastacken	`STACK ?`	`STACK ?`
`BEGIN DROP AGAIN`, tömmer den	`STACK ?`	`STACK ?`
`BEGIN DROP DROP 1 AGAIN`	`STACK ?`	`STACK ?`, BASIC klarar sig
`10 0 DO R> DROP LOOP`, returstacken	`STACK ?`	hänger sig

Att datastacken fylls fångar båda. Att den töms fångar varianten bara därför att SP går runt minnet. `DROP` tar ett värde i taget, och när SP har gått förbi `$FFFF` och börjar om på 0 ligger den under BASIC:s variabler, så prövningen slår till. Att BASIC klarade sig i det tredje provet är tur snarare än skydd, eftersom slingan har läst och skrivit hela vägen runt. En slinga som tömmer returstacken fångas inte alls. `R>` tar då BASIC:s egen stack som returadresser, och när det inte finns några anrop i slingan prövar inte heller `DOCOL` något.

Att pröva stackarna bara mellan orden som skrivs in är vad fig-FORTH gjorde. Dess `INTERPRET` anropar `?STACK` efter varje ord som tolkas, men inte inne i kolonorden. Bandets tolk är försiktigare, eftersom en Forth som körs från BASIC annars kan ta BASIC-programmet med sig. Varianten ligger mitt emellan: den skyddar BASIC-programmet mot den vanligaste olyckan, en slinga som lägger något på stacken varje varv, men inte mot de andra.

3.6 Vad mer hade gått att vinna

Inget av det här är prövat. Det är räknat ur listningen på samma sätt som tiderna ovan, och de räkningarna har stämt med klockan.

Prövningen i `DOCOL`. Den kostar 97 T per anrop, mer än `DOCOL` utan den. Utan den hade slingan med `NOLL` tagit 474 T per varv i varianten, omkring 1,58 s, och den slingan hade gått omkring 17 % fortare. Men den prövningen går inte att göra billigare på samma sätt som i `BRANCH`. Returstacken har bara de 64 byte som ligger mellan BASIC:s stack och datastacken, och en prövning på den höga byten hade slagit till långt för tidigt eller inte alls. Det som återstår är att stryka den, och då stoppas en rekursion som inte tar slut först när den har skrivit över kanariefågeln och datastacken, och kanske inte förrän BASIC:s stack är borta.

`NEXT` *i varje ord.* Varje ord i maskinkod slutar med `JP NEXT`. Med de sju byten i `NEXT` upprepade i varje ord sparas 10 T per ord. I raden `I I + 5 - A !` är det omkring en tiondel av varvet. Kärnan har 48 sådana hopp, så tolken hade blivit omkring 190 byte större, och en tolk som ska rymmas i en krets betalar för varje byte. Det vanliga greppet på Z80 är att hålla `NEXT`:s adress i IY och sluta varje ord med `JP (IY)`, 8 T och 2 byte, men här hör IY till ROM:en.

Andra sätt att tråda. Indirekt trådad kod, som fig-FORTH använde, har en adress till i varje steg och blir långsammare. Subrutintrådad kod, där tråden är en följd av `CALL` och varje ord slutar med `RET`, kostar 27 T per steg i stället för 48. Men då måste Z80:s stack vara returstacken, och datastacken flyttas till ett annat register. Varje `PUSH` och `POP` på datastacken hade då blivit flera instruktioner med IX, och de är fler än stegen. Tokentrådad kod, där ett steg är en byte som slås upp i en tabell, är det som BASIC:en gör, och den kostar mer per steg men tar mindre plats.

Det som redan är gjort. Att det senast lagda värdet ligger i BC sparar en `POP` och en `PUSH`, 21 T, i nästan varje ord som räknar. Det är troligen den största enskilda vinsten i tolken, och den kostar ingenting i säkerhet.

Det allmänna i försöket är att det som görs i varje steg är det som räknas. En trådad tolk är snabb därför att steget mellan orden är kort, och allt som läggs i det steget eller i hoppen betalas i varje varv. En prövning som görs en gång per rad kostar nästan ingenting, och samma prövning i `BRANCH` kan kosta mer än slingan själv. Frågan är därför inte bara hur en prövning görs billigare, utan var den behöver ligga. I en tolk som delar minne med BASIC har varje svar ett pris i säkerhet.

3.7 Inte prövat

Bara ordet `BRANCH` är ändrat. En `DOCOL` utan prövning, `NEXT` i varje ord och andra sätt att tråda är räknade men inte byggda. Tiderna är emulatorns och inte prövade mot en riktig ABC80.

Skripten ligger i `utanfor/forth_snabb` i kodförrådet. `gor.sh` bygger varianten ur en kopia av bandets källor, och `matning.sh` kör proven i BASIC och i båda tolkarna.

COMAL 0,5

Utanför banden. Det som ändras är en kopia av ABC80-ROM:en (11273, den som band 1 beskriver) med 23 byte ändrade och en krets på `$4000`–`$48A6`, 2 215 byte. Tolken får COMAL:s styrstrukturer: WHILE, REPEAT, IF på flera rader, procedurer med parametrar och lokala variabler, CASE och LIST med indrag. Strukturerna mäts i emulatorn mot samma program med GOTO och GOSUB och mot BASIC II:s WHILE och WEND. ABC80-BASIC:en fick aldrig något av det här.

Jan Boström arbetade på Liber och sedan på Luxor. I en intervju 2008 säger han om skoldatorn Compis och dess språk COMAL: ”Man borde ha fortsatt med BASIC-tolken istället, byggt vidare på det konceptet.” Han lägger till att man inte kunde veta det då. Frågan här är hur mycket det hade kostat. Hade COMAL:s strukturer fått plats i ABC80-BASIC:en, utan att den blev ett annat språk och utan att gamla program slutade fungera? Och vad hade de kostat i tid mot de GOTO och GOSUB som de ersätter?

Försöket svarar med en krets. Den har samma storlek och plats som de tilläggskretsar som fanns, och ROM:en får bara de hopp som behövs för att hitta den. Den gamla tolkens egna rutiner gör det mesta av arbetet. Allt är vanlig Z80-kod som hade gått att skriva 1978. Bilagan COMAL 0,5 på ABC80 visar satserna i ett program som körs på emulatorns webbsida.

4.1 COMAL och Compis

COMAL gjordes i Danmark av Børge R. Christensen, lärare i matematik på Tønder Seminarium, och Benedict Løfstedt vid Aarhus universitet. Det första COMAL kom 1975. Tanken var att behålla det som gjorde BASIC lätt, radnumren, direktläget och tolken, och ta med det som gjorde Pascal tydligt: block som stängs med sitt namn, procedurer med parametrar och lokala variabler och långa variabelnamn. Danska skolor krävde att COMAL skulle finnas på varje mikrodator de köpte, och därför fanns det COMAL för PET, Commodore 64, RC Piccolo, BBC Micro och många andra.

I Sverige startade Styrelsen för teknisk utveckling 1981 en upphandling, TUDIS, av en svensk skoldator, som skulle ersätta ABC80. Den blev Compis, med Intel 80186, CP/M-86 och COMAL, byggd av Esselte och Svenska Datorer och sedan Televerkets Telenova. Den blev minst ett år försenad, dyrare än budet och såldes bara till skolor. Compis brukar räknas som ett av de stora misslyckandena i svensk datorhistoria. Det var knappast språket som var felet. Det var en dator som skulle konstrueras från början och mötte IBM PC och MS-DOS, och som ingen hade hemma.

Försöket säger alltså inget om att Compis hade kunnat undvikas. TUDIS gällde en ny dator, inte ett nytt språk. Det prövar bara Boströms tanke: att COMAL:s strukturer hade kunnat komma som en krets till de datorer som redan stod i skolorna.

4.2 Satserna

Det är fortfarande ABC80-BASIC. Radnumren, GOTO och GOSUB, alla gamla satser, de korta variabelnamnen och BCD-talen står kvar. De nya satserna är dessa:

Satserna	Fel
`WHILE v ... ENDWHILE`	70: ett block stängs aldrig
`REPEAT ... UNTIL v`	71: en stängning utan början, eller av fel slag
`IF v THEN ... ELIF v THEN ... ELSE ... ENDIF`	70, 71
`PROC namn(a,b%) ... ENDPROC`	72: `PROC` saknas, fel antal eller typ
`EXEC namn(x,y%)`	72
`LOCAL a,b%`	72: inte först efter `PROC`
`CASE v OF, WHEN a,b, OTHERWISE, ENDCASE`	73: ingen `WHEN` stämmer; 17: sträng mot tal

Ett `IF` med ingenting efter `THEN` öppnar ett block. Med en sats efter `THEN` är det ett vanligt `IF`, som förut. Parametrarna och `LOCAL` har grund bindning: `EXEC` lägger variablernas gamla värden på stacken och `ENDPROC` lägger tillbaka dem. Variablerna behåller alltså sina adresser, och rekursion fungerar. Typen ska stämma redan i anropet, så `EXEC P(5)` till en heltalsparameter ger fel 72, och det ska vara `5%`.

Fel i strukturen syns före körningen. Blocken paras vid `RUN`, som `FOR` och `NEXT`, och ett `WHILE` som aldrig stängs ger fel 70 innan första raden körs. `LIST` drar in det som står inom ett block två steg per nivå. Ett program som flyttar Hanois torn med en rekursiv procedur ser ut så här efter `LIST` och `RUN`:

```
10 EXEC HANOI(3,1,3,2)
20 PRINT
30 END
100 PROC HANOI(N,A,B,C)
110   IF N>0 THEN
120     EXEC HANOI(N-1,A,C,B)
130     PRINT A;"-";B;
140     EXEC HANOI(N-1,C,B,A)
150   ENDIF
160 ENDPROC

ABC80

 1- 3 1- 2 3- 2 1- 3 2- 1 2- 3 1- 3

ABC80
```

Skärmen är emulatorns. Provprogrammet skriver in raderna och kommandona direkt, så `LIST` och `RUN` syns inte.

Det som gör COMAL till ett eget språk är inte med: långa variabelnamn, funktioner på flera rader (`FUNC`) och parametrar som tar variabeln i stället för värdet (`REF`). Strängar går inte heller som parametrar. Långa namn skulle kräva en ny variabeltabell och en ny länkning, och då är det inte längre samma tolk.

4.3 Hur det är byggt

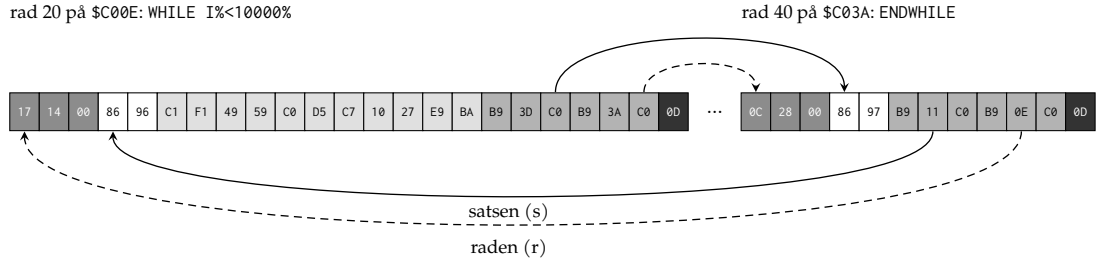
ROM:en har två tabeller med satser. Satserna i den andra skrivs som $86 följt av en kod, och band 1 beskriver dem i kapitlet Satserna kompileras. Tabellen slutar på $95, men koden räcker till $FF. Kretsen har kopior av tabellens tre delar, nyckelorden, kompileringen och körningen, med de nya satserna efter de gamla, $96–$A7. De flesta lapparna i ROM:en byter bara en adress, så att ROM:en läser kopiorna:

Adress	Rutin	Lappen	Byte
$0AF7	LIST	radräknaren och indraget sätts i kretsen	4
$0B5F	LIST	raden skrivs med indrag	2
$0D59	RUN	förlänkningen går genom kretsen	2
$0DB4	EXEST2	körningen av tabell 2	2
$0EC0	LNKELM	$86 med de nya koderna länkas i kretsen	3
$11D6	DETOK	nyckelorden som text	2
$18E1, $18E7	TOKKW	nyckelorden och kompileringen	4
$1A18	tabell 1	IF kompileras i kretsen	2
$1BD3	CMPDMI	DIM-felet lagat (se nedan)	2

Lappen i LNKELM tar platsen för en prövning av $8D, som ger fel 8 men aldrig står i koden. Kretsen ligger på $4000, där Superbasic låg.

En sats som behöver veta var en annan del av blocket står får två platser i koden, $B9 följt av en adress. Det är samma element som FOR och NEXT har för sitt block, och SKPELM, LNKELM och DETOK hoppar redan över det. Den ena platsen, s, är adressen till den andra satsens $86. Den andra, r, är adressen till dess rad. Kompileringen skriver tomma platser.

Vid RUN fyller länkningen i dem. PRELNK går igenom programmet som förut, och kretsen tar de nya satserna. Medan ett block är öppet håller dess s-plats det förra öppna blocket, och adressen till det senast öppnade står i två byte först på heapen. De öppna blocken bildar alltså en stack i själva koden, utan eget minne. När ENDWHILE länkas tas WHILE från stacken, och de två satserna får varandras adresser. Står något kvar på stacken när programmet är slut blir det fel 70. Figuren visar två rader ur ett mätprogram, läst ur minnet efter RUN.



Raderna 20 WHILE I%<10000% *och* 40 ENDWHILE *efter länkningen, läst ur minnet i emulatorn. Mörkgrått är radens längd och nummer, vitt satsens* $86 *och kod, ljust villkoret, grått platserna* $B9 *med sin adress och svart CR. Varje sats har två platser: s pekar på den andra satsens* $86 *(hela pilar) och r på dess rad (streckade). Rad 30 står mellan dem.*

Under körningen följer satserna platserna och behöver ingen stack. Ett falskt WHILE läser s och fortsätter efter ENDWHILE. Därför kan GOTO hoppa ut ur WHILE, REPEAT, IF och CASE utan att något blir kvar. EXEC lägger återhoppet på stacken som GOSUB. Ett PROC som nås i den vanliga följden hoppar förbi sin ENDPROC. På samma sätt hoppar en WHEN eller OTHERWISE till efter ENDCASE när delen före är klar. Ett hopp till en annan rad går samma väg som RUNLIN, så TRACE skriver radnumret som för GOTO.

Kretsens 2 215 byte fördelar sig så här:

Del	Byte
nyckelorden och tabellerna, med kopian av tabell 2	369
kompileringen	313
länkningen vid `RUN`	574
körningen, varav `CASE` och `WHEN` 317	725
`LIST` med indrag	234

BASIC II behövde 30 kilobyte ROM, men gav också långa variabelnamn, binära flyttal och mycket annat.

4.4 Tiderna

Proven är små program i samma par: ett med GOTO eller GOSUB och ett med den nya strukturen. S-proven räknar till 10 000 med heltal, A-proven lägger ihop 1 till 2 000 genom ett anrop, och V-proven väljer mellan fyra fall 2 000 gånger. Tiden är emulatorns, i sekunder från `RUN` till `READY`. BASIC II är med där den har en motsvarighet.

Prov	Vad	ABC80	COMAL	BASIC II
SFOR	`FOR` och `NEXT`	3,01	3,01	2,90
SGOTO	`IF I%<10000% THEN 20`	11,52	11,52	9,35
SWHILE	`WHILE` och `ENDWHILE`		12,10	9,76
SREPEAT	`REPEAT` och `UNTIL`		11,91	
AGOSUB	`X=I` och `GOSUB`	6,94	6,94	5,51
AEXEC	`EXEC ADD(I)`		9,36	
ALOCAL	som AEXEC med `LOCAL T`		11,76	
VIF	`IF ... : GOTO`	27,30	27,30	14,75
VBLOCK	`IF`, `ELIF`, `ELSE`		27,98	
VCASE	`CASE` och `WHEN`		28,50	

BASIC II:s SWHILE har `WEND`. Där strukturerna ersätter GOTO kostar de lite. `WHILE` är 5,0 % långsammare än samma slinga med `IF` och `GOTO`, `REPEAT` 3,4 %, block-`IF` 2,5 % och `CASE` 4,4 % mot en kedja av `IF`. I BASIC II kostar `WHILE` och `WEND` 4,4 % mot dess egen slinga med `GOTO`, alltså ungefär lika mycket. VIF går nästan dubbelt så fort i BASIC II för att `I-INT(I/4)*4` räknas med binära flyttal, inte för att `IF` är snabbare (kapitlet Två tolkar, samma förbättringar?).

Procedurerna kostar mer. `EXEC` med en parameter tar 35 % längre än `GOSUB` med en tilldelning, omkring 3 600 T mer per anrop, och `LOCAL` lägger till en fjärdedel till. Där byts variablernas värden på stacken, både in och ut, och `LOCAL` är dessutom en egen rad.

4.4.1 ENDWHILE räknar villkoret

Det första `ENDWHILE` hoppade tillbaka till `WHILE`, som räknade villkoret. Då var SWHILE 14 % långsammare än SGOTO, 13,1 s. Nu räknar `ENDWHILE` villkoret själv. Är det sant hoppar den till satsen efter `WHILE`, och är det falskt fortsätter den utan hopp. `WHILE` körs alltså bara en gång, och varje varv slipper en sats. Så här ser det ut:

```
44FF  EB           ENDWX:  EX   DE,HL          HL efter $86 $97
4500  CD EF 43             CALL RDSLOT        DE = WHILE, BC = dess rad
4503  E5                   PUSH HL            efter platserna
4504  13                   INC  DE
4505  13                   INC  DE            villkoret
4506  C5                   PUSH BC
4507  FF                   RST  $38           räknas, värdet i HL
4508  C1                   POP  BC
4509  7C                   LD   A,H
450A  B5                   OR   L
450B  E1                   POP  HL
450C  20 02                JR   NZ,SKIP6H     sant: ett varv till
450E  EB                   EX   DE,HL         falskt: vidare efter
450F  C9                   RET                ENDWHILE
4510  21 06 00     SKIP6H: LD   HL,$0006
4513  19                   ADD  HL,DE         förbi WHILE:s platser
4514  EB                   EX   DE,HL
4515  18 48                JR   HOPP          till raden i BC
```

RDSLOT läser båda platserna. RST $38 är ROM:ens uttrycksberäkning, och den lämnar DE efter villkoret, där WHILE:s platser står. HOPP gör som RUNLIN om raden är en annan än den som körs och flyttar annars bara DE. Med den här vägen tar SWHILE 12,1 s. Det är omkring 170 T mer per varv än SGOTO, som också räknar ett villkor och hoppar en gång per varv.

4.4.2 Platsen först

I CASE, WHEN och ELIF står platserna före uttrycket, inte efter som i WHILE. Ett WHEN som inte stämmer följer s till nästa WHEN, och ett block-IF vars villkor är falskt följer s till nästa ELIF. Står platsen först behöver satsen inte gå förbi värdena eller villkoret för att hitta den. Med platserna efter tog VCASE 37,0 s och VBLOCK 29,6 s.

ELIF är därför två satser i koden. Den första, med platserna, körs när delen före är klar och hoppar till ENDIF. Den andra, $86 $9F, är villkoret, och dit hoppar ett falskt villkor före.

4.4.3 CASE utan kopior

Det första CASE byggde koden för ”väljaren värdet =” på stacken för varje WHEN-värde och lät RST $38 räkna den. Då jämfördes alla tre typerna som i ROM:en, men byggandet krävde uttryckens längder, och ENDTOK gick igenom uttryckskoden för att hitta dem varje gång. VCASE tog 32,1 s.

Nu räknas väljaren en gång och ligger kvar på stacken. Varje WHEN-värde räknas där det står i programmet, och RST $38 lämnar DE efter det, så ingen längd behövs. Värdet jämförs med väljaren: två byte för heltal, ROM:ens FCMP för flyttal och längden och tecknen för strängar. Ett heltal mot ett flyttal görs om till flyttal med INTTOF i fem byte i CASE:s del av stacken. En sträng som räknas fram, med CHR$ eller +, ligger kvar på stacken tills den är jämförd. VCASE tar 28,5 s.

Det som återstår av de 4,4 % är mest flyttalen. Ett flyttalsvärde i WHEN slutar med $B9, och ENDF flyttar då återhoppet under talet. IF:s jämförelse slutar med ett heltal och slipper det. CASE blir snabbare än kedjan av IF när väljaren är ett längre uttryck, som räknas en gång i stället för en gång per IF. En hopptabell vore snabbast för heltal, men den kräver att alla värden är konstanter.

4.5 Gamla program

De 16 proven från kapitlet Två tolkar, samma förbättringar? körs med och utan kretsen och ger samma skärm, samma fel och samma tal. Tiden för själva körningen är densamma. Varje `RUN` tar 417–473 T längre, för länkningen går genom kretsen och lägger stackens två byte först på heapen. Det är omkring 0,15 ms.

Kretsens egna 13 prov går igenom alla satserna, felen, `LIST`, `TRACE`, rekursionen och `CASE` med alla tre typerna, och ger rätt skärm.

4.6 Vad som kom fram på vägen

– *Tabell 2 slutar på* `$95`. ROM:ens andra tabell har 22 satser, `$86 $80` till `$86 $95`, men ingenting i koden hindrar fler. Det räcker att flytta tabellerna. Nyckelorden söks i tur och ordning, så de nya som börjar med END står före `END`, annars hittar `KWSRCH END` i `ENDWHILE`.
– *DIM-felet.* När `DIM` går fel i `CMPDMI` ligger ett registerpar kvar på stacken, och felrutinen hoppar till en adress som beror på variabelns namn (band 1, bilagan Fel och egenheter i tolken). I kapitlet Två tolkar, samma förbättringar? flyttades kretsen till `$5000` för att undvika det. Här lagas felet i stället: `DIMFIX` tar bort registerparet och går till felet som det var tänkt.
– *LIST pausar efter 22 rader.* Proven listar därför bara korta program, och lappen på `$0AF7` sätter radräknaren som förut.

4.7 Inte prövat

Kretsen är bara prövad i emulatorn, inte i en riktig ABC80. Program sparade med de nya satserna som BAC-filer är inte prövade. Platserna skrivs om vid varje `RUN`, så de borde gå. `LIST` till en fil räknar indraget, men bara skärmen är prövad.

Koden ligger i `utanfor/comal` i kodförrådet: `comal.asm` med satsernas form överst, `bygg.py` som skriver ROM:en och kretsen, `prov.sh` med proven och `matt.sh` med mätningen. `BAKGRUND.md` har källorna till avsnittet om COMAL och Compis. På webbsidan finns kretsen med `?krets=comal`, som också byter till den lappade ROM:en.

Forthaid

Utanför banden. Forthaid är en verktygskrets i Smartaid 3:s stil på `$4000`–`$5FFF`, 8 176 byte, skriven i Forth på tolken från exempel 7 i band 2. Tolken är en kopia där bara `BYE` och ordlistans sökning är ändrade, och bokens filer är orörda. Kommandona mäts i emulatorn mot Smartaid 3 på samma program. Ingen kod är lånad från Smartaid; kapitlet om Smartaid i band 2 är bara använt för att veta vad kommandona gör och hur kretsarna tar över tangentbordet. En sådan krets fanns aldrig.

I band 2 slutar exempel 7, den lilla Forth-tolken, med en tanke (kapitlet Egna exempel, avsnittet Ett öppet alternativ). Smartaid var sluten: det som fanns i kretsen var det som hade lagts dit från början. En Forth på `$4000` hade i stället varit öppen, och användaren hade kunnat lägga till egna ord. Tanken prövas här genom att skriva Smartaid i Forth. Får en sådan krets plats på samma 8 kilobyte? Hur mycket långsammare blir kommandona? Och vad hade behövts för att de skulle gå lika fort?

Svaret är en krets med Smartaid 3:s sätt att ta över tangentbordet, en skärmeditor, stegning av ett program som kör och tretton kommandon för programmen. Den går i båda ROM-versionerna, 9913 och 11273. Bilagan Forth på ABC80 är en kort genomgång av Forth i Forthaid, från stacken till ett eget kommando för BASIC-raderna.

5.1 Så används den

Kretsen startas på tre sätt:

```
Z=CALL(16384)      Forth-tolken, med kommandona som ord
Z=CALL(16384,1)    samma, med de egna ord som redan finns
Z=CALL(16384,2)    kretsen kopplas in, kommandona direkt i BASIC
```

Efter det tredje tar kretsen över varje gång BASIC:en står i direktläge och väntar på en rad. Raden skrivs med kretsens skärmeditor, där markören går fritt över skärmen och högerpilen läser in tecknet under den, som i Smartaid. Börjar raden med ett av kretsens kommandon kör kretsen det. Andra rader, programrader och direkta satser, går till ROM:en som förut. `EX` kopplar ur kretsen, och då är ABC80 som förut.

Kommandona är dessa:

Kommandot	Vad
`LIST` *n-m*	listar, med paus efter 22 rader
`FIND` *text*	de rader där texten står, så som raden listas
`VAR`	alla variabler i programmet, i bokstavsordning
`VAR` *namn*	de rader där variabeln står
`CHANGE` *v1,v2*	byter en variabel mot en annan i hela programmet
`REN` *a,s,n-m*	numrerar om, också en del av programmet
`DEL` *n-m*	tar bort raderna
`AUTO` *a,s*	skriver radnumren
`ED` *n*	listar raden och ställer markören på den
`START` *n*	kör från raden utan att nollställa variablerna
`OLD`	tar tillbaka ett program efter `NEW`
`PEEK` *adress*	minnet åtta byte per rad, i hex och som tecken
`SYS`	programmets gränser och det lediga minnet
`HELP`	kommandona och de egna orden
`FORTH`, `EX`	till Forth-tolken, och ur kretsen

I tolken är kommandona vanliga Forth-ord, och de kan skrivas med versaler eller gemener. Ett eget ord kan använda kretsens ord för BASIC-raderna, som `BOFA`, `NEXTLINE`, `END-PRG?` och `LISTLINE`, och är sedan ett kommando som de andra. De egna orden ligger i kassettens blockbuffertar, 304 byte på `$FB50`–`$FC7F`. Det räcker till några dussin korta ord.

Smartaid 3 är också 8 kilobyte, men har plats för arton kommandon och en skrivarrutin. Forthaid saknar `RESUME`, `DIR`, `LIB`, `FOR` i direktläge och `PR:`. I stället har den `VAR` med namn och en tolk som går att bygga ut.

5.2 Hur den tar över

Forthaid tar över som Smartaid (band 2, kapitlet Smartaid, Superbasic och ABC80L, avsnittet Hur Smartaid tar över). `Z=CALL(16384,2)` sätter I-registret till `$5F`, så att tangentbordets vektor läses ur kretsen på `$5F34`. Där står adressen till `KBDINT`, och därefter kassettens, som pekar på ROM:ens egen rutin. Vektortabellen måste alltså ligga på just `$5F34`, och byggskriptet prövar det. `EX` sätter I till 0 igen.

Avbrottet ska bara ta över när ROM:en står i kommandoloopen och väntar på en rad. Kretsen prövar att `$FE24` är 1, alltså direktläge, och tittar sedan på stacken. När kommandoloopen väntar ligger radbuffertens adress `$FE40` närmast `STKTOP` och under den återhoppet `$00F0` från radläsningen:

```
5F40  3A 24 FE        LD   A,($FE24)      1 i direktläge
5F43  3D              DEC  A
5F44  20 25           JR   NZ,KBDRUN      ett program kör
5F46  2A 27 FE        LD   HL,($FE27)     STKTOP
5F49  2B              DEC  HL
5F4A  7E              LD   A,(HL)
5F4B  FE FE           CP   $FE            $FE40
5F4D  20 2A           JR   NZ,KBD1
5F4F  2B              DEC  HL
5F50  7E              LD   A,(HL)
5F51  FE 40           CP   $40
5F53  20 24           JR   NZ,KBD1
5F55  2B              DEC  HL
5F56  7E              LD   A,(HL)
5F57  A7              AND  A              $00F0
5F58  20 1F           JR   NZ,KBD1
5F5A  2B              DEC  HL
5F5B  7E              LD   A,(HL)
5F5C  FE F0           CP   $F0
5F5E  20 19           JR   NZ,KBD1
5F60  2A 27 FE        LD   HL,($FE27)     kommandoloopen:
5F63  F9              LD   SP,HL          stacken töms
5F64  21 7E 5F        LD   HL,KBD2
5F67  E5              PUSH HL
5F68  C3 1E 03        JP   KEYINT         ROM:ens tangentrutin
```

Före det har rutinen läst port $38 och gått till stegningen om tangenten har släppts. Stämmer inte de fyra byten går avbrottet vidare till ROM:ens KEYINT på KBD1, som om kretsen inte fanns. Stämmer de sätts SP till STKTOP, och ROM:ens tangentrutin lägger tangenten i sin buffert som vanligt. Den hoppar sedan tillbaka till KBD2 i stället för till kommandoloopen. Där får Forth sina stackar på $FC80–$FCFF, och tolken startas på sin fjärde ingång, KBD, som läser raden med skärmeditorn till ROM:ens radbuffert på $FE40. Den första tangenten ligger redan i tangentbufferten. Är raden ett kommando körs det. Annars sätter TOROM stacken som kommandoloopen hade den, med $FE40 vid SP, och hoppar in efter radläsningen på $00F3. ROM:en tar då raden som om den hade läst den själv.

Prövningen hänger på att 9913 och 11273 har samma återhopp, $00F0. En annan ROM-version, med kommandoloopen på en annan adress, skulle inte kännas igen.

5.3 Stegningen

Medan ett program kör ger CTRL-C en paus i stället för STOP. Programmet står still, och ett utropstecken står där markören är. Nästa tangent bestämmer. CTRL-T kör programmet med TRACE så länge tangenten hålls ned, CTRL-S kör det utan, och CTRL-C stoppar det som vanligt. När tangenten släpps blir det paus igen. Ett kort tryck kör alltså några rader.

Svårigheten är att märka när tangenten släpps. Tangentbordets PIO ger normalt avbrott bara när en tangent trycks ned. Bit 7 i port $38 är 1 så länge den hålls. Pausen ställer därför om PIO A: med styrordet $97 kommer avbrottet när tangenten släpps, och med $B7, som ROM:en ställer det, när den trycks ned. Medan CTRL-T eller CTRL-S hålls står $97 kvar och en flagga, STEPF, är satt. Avbrottet som kommer när tangenten släpps ställer tillbaka $B7 och går direkt till pausen. Det är 212 byte i assembler, med vektortabellen.

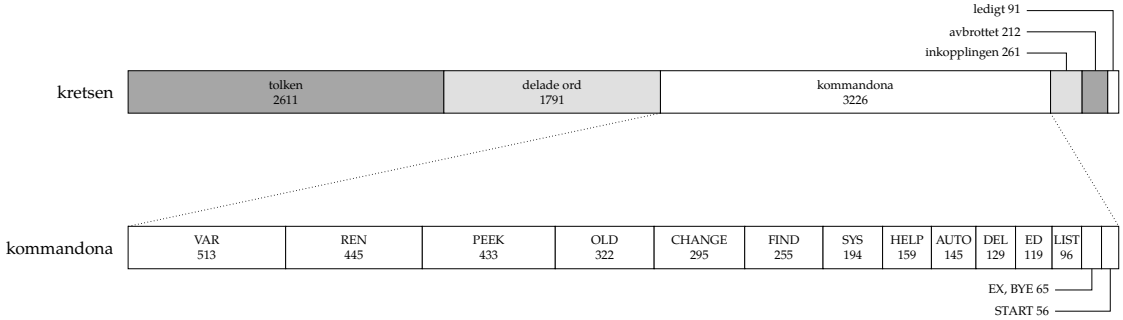
Stegningen hade ett fel som provet fann först när tiderna i det flyttades. Släpps tangenten mitt i ROM:ens TSTBRK, mellan läsningen av BRKFLG och nollningen, nollar TSTBRK efter pausen den flagga som CTRL-C i pausen har satt. Programmet går då vidare i stället för att stanna. Det hände vid 3–4 av 161 hålltider mellan 40 och 200 ms. Nu prövar pausen återhoppet på stacken (PSBRK,

22 byte). Ligger det i fönstret `$033E–$0344` sätts det till `$033E`, så att `TSTBRK` läser flaggan igen. Med det stannar programmet vid alla 161 hålltiderna, i båda ROM-versionerna.

För att stegningen skulle kunna provas fick emulatorerna två saker de inte hade. PIO A:s avbrott följer bit 5 i styrordet, så att `$97` ger avbrott när tangenten släpps. Provets tangenter kan också hållas nere en angiven tid.

5.4 Minnet

Figuren visar hur kretsens 8 192 byte fördelar sig, räknat ur listningen. Ett ord som bara ett kommando använder räknas till det kommandot.



Forthaids 8 192 byte efter vad de gör. Mörkt är tolken från band 2 och avbrottet i assembler, ljust orden som flera kommandon delar och inkopplingen med radläsningen. Nedtill kommandona för sig, med `EX` *och* `BYE` *tillsammans. Det är ingen minneskarta, för orden ligger om varandra i kretsen.*

Bokens tolk, utan exempelorden, är 2 611 byte, alltså nästan en tredjedel. Orden som flera kommandon delar är 1 791 byte, och bland dem finns skärmeditorn. 75 byte är lediga före vektortabellen och 16 efter avbrottet. Det som ska läggas till måste rymmas där eller ersätta något.

Forthaid lånar inget av BASIC:ens minne. Med och utan kretsen tar ett program slut på samma adress. Kretsen använder dessa platser:

Var	Vad
`$7C78–$7EFE`	tolkens och kommandonas variabler, i bildminnets osynliga byte
`$7F7E`	`STEPF`, satt medan CTRL-T eller CTRL-S hålls ned
`$FB00–$FB4F`	radbufferten
`$FB50–$FC7F`	de egna orden, 304 byte
`$FC80–$FCFF`	stackarna medan kretsen läser en rad och kör ett kommando

`$FB00–$FCFF` är kassettens blockbuffertar, och ROM:en använder dem bara medan en fil läses eller skrivs. Då läser kretsen ingen rad. De egna orden skrivs däremot över av en kassettfil. Stackarna låg först under BASIC:ens SP. När ett program fyllde minnet skrev de över dess slut, och tolken svarade `STACK ?` om och om igen. 128 byte räcker med marginal. På ett program om 12 kilobyte gick returstacken som djupast 20 av 64 byte och datastacken 46 av 62, mätt genom att fylla området med `$AA` före kommandot.

5.5 Tiderna

Kommandona mäts på tre program, 2, 8 och 12 kilobyte långa, med Forthaid och med Smartaid 3 i emulatorn och ROM:en 9913. Tiden är den emulerade maskinens, i sekunder, minus tiden för ett program med två rader. `FIND` letar efter en variabel som bara står på den sista raden. `VAR` visar alla 81 variabler, och `VAR Z9` raden där Z9 står, vilket Smartaid 3 inte kan. `REN 10,10` numrerar om allt, och `CHANGE` byter B1, som står på var åttonde rad, mot Q1.

Programmet	Kretsen	FIND	VAR	VAR Z9	REN	CHANGE
2 060 byte	Forthaid	0,35	0,60	0,18	0,41	0,18
	Smartaid 3	0,08	0,21	–	0,23	0,14
8 180 byte	Forthaid	1,50	1,05	0,74	2,21	0,72
	Smartaid 3	0,42	0,74	–	1,60	0,62
12 266 byte	Forthaid	2,29	1,36	1,12	3,90	1,09
	Smartaid 3	0,66	1,10	–	3,00	0,93

Forthaid är 3,5–4,5 gånger långsammare med `FIND`, 1,2–3 gånger med `VAR`, 1,3–1,8 med `REN` och 1,2–1,3 med `CHANGE`. Skillnaden krymper när programmet växer, eftersom en del av den är fast. `REN` ligger närmare Smartaid 3 än `FIND`, för där gör ROM:ens egna rutiner det mesta, `LNKREF` och `REN`:s tredje varv.

De första versionerna var mycket långsammare. Där arbetet var en slinga i Forth över programmets byte var Forthaid 10–15 gånger långsammare än Smartaid 3, och `FIND` och `VAR` tog sekunder där Smartaid tar tiondelar. Tre maskinkodsord tog de heta slingorna.

5.5.1 FIND

Exempel 7 har ett `FIND` som söker i radens kod, med `LINE?` och `MATCH` i Forth. Forthaids `FIND` söker i raden så som den listas, så att nyckelord, radnummer och variabler hittas som de står på skärmen. Varje rad görs först till text med `DETOK`, som använder ROM:ens tabell. Sedan jämförde en `DO`-slinga i Forth texten med det sökta, en gång för varje läge i raden. Det tog 0,98, 4,19 och 6,37 s. Nu gör maskinkodsordet `SEARCH` hela jämförelsen:

```
: HIT?  ( a -- f )  \ står PAT PLEN i raden, så som den listas?
  HEAP @ DETOK  HEAP @ -  1-  HEAP @ SWAP PAT @ PLEN @ SEARCH ;
: FIND
  REST PLEN ! PAT !  CR PAGE  PLEN @ 0= IF EXIT THEN
  BOFA @ BEGIN DUP END-PRG? 0= WHILE
    DUP HIT? IF DUP LISTLINE MORE? IF DROP EXIT THEN THEN
    NEXTLINE
  REPEAT DROP ;
```

`DETOK` skriver raden på heapen och lämnar adressen efter den. `SEARCH` tar texten och det sökta och svarar sant eller falskt. `FIND` blev 2,8 gånger snabbare och dessutom kortare. Det som är kvar är mest `DETOK` och slingan över raderna. Troligen är det `DETOK` som gör `FIND` 3,5 gånger långsammare än Smartaid 3, men det är inte mätt.

5.5.2 VAR

Det första `VAR` höll en lista över de namn det redan hade sett och letade igenom den, i Forth, för varje förekomst. Tiden växte med antalet förekomster gånger antalet namn: 3,06, 10,77 och 16,30 s. Nu har varje tänkbar variabel en bit i en tabell på heapen. Det är 26 bokstäver, 11 siffror (ingen och 0–9), 3 typer och med eller utan index, 1 716 bitar i 215 byte. Maskinkodsordet `SETVARS` går igenom en rad och sätter biten för varje variabel. Platsen räknas ut ur namnbyten och bokstaven, så varje förekomst kostar lika lite hur många namn det än finns. Sedan skrivs tabellen ut i bitarnas ordning, och tomma byte hoppas över. Listan kommer därför sorterad, A, A(, A1, utan att något

sorteras. Det gjorde VAR omkring tio gånger snabbare. Tabellen behöver 256 byte mellan heapen och STKTOP, och annars svarar VAR ?.

CHANGE och VAR med namn fick på samma sätt MATCHVAR, som letar upp nästa förekomst av den sökta variabeln direkt i stället för att ge varje variabel tillbaka till Forth för jämförelse. Båda blev omkring 25 % snabbare. MATCHVAR och SETVARS går igenom raden med samma rutin, som hoppar över elementen och uttryckskoden med ROM:ens SKPELM och SKPEXP. Den genomgången är nu det mesta av tiden, och Smartaid 3:s VAR tar ungefär lika lång tid.

5.6 Orden i CMOS-minnet

Band 2 föreslår i samma avsnitt att de egna orden kunde ha sparats i Super Smartaids CMOS-minne med batteri på $5000–$57FF. Det är prövat för sig, på exempel 7:s tolk utan Forthaids kommandon. Tolken ligger på $4000, och ordlistan börjar på $5008 och växer uppåt. Radbufferten ligger överst, på $57B0, så tolken behöver inget RAM från BASIC:en. De första åtta byten håller det som behövs för att hitta orden igen: det sista huvudet i kretsen, var nästa ord börjar, det senaste huvudet och en summa av de tre plus $A5A5.

Efter varje rad som gick utan fel, och utan en halvfärdig definition, skrivs de tre och summan. Vid start prövas märket och summan. Stämmer de inte, för att minnet är tomt eller kretsen en annan version, börjar tolken med en tom ordlista och skriver NY ORDLISTA. Det är Super Smartaids CMOS JEOPARDIZED i liten skala. Så här ser den första starten ut i emulatorn:

```
Z=CALL(16384,3)
NY ORDLISTA
: DUBBEL DUP + ; OK
21 DUBBEL . 42 OK
BYE

ABC80
```

Sedan startas emulatorn om med samma fil som CMOS-minne, som när datorn slås av och på. Efter Z=CALL(16384,3) kommer ingen NY ORDLISTA, och 21 DUBBEL . ger 42 direkt.

Det ger 1 960 byte till egna ord, mer än Forthaids 304. Forthaid självt fyller $4000–$5FFF och kan inte ha CMOS-minnet på $5000. En Forthaid med sparade ord hade behövt en krets till, eller bankning. Forthaid II i nästa kapitel har bankning och sparar i stället orden på skiva. Den fasta hopptabell som band 2 säger att sparade ord skulle behöva finns inte här. Därför prövas märket: ord som sparats med en annan version av kretsen glöms hellre än körs.

5.7 Begränsningarna

- *Ingen automatisk start.* Smartaid 3 kopplar in sig genom DOS-kretsens plats för kallstart. Forthaid har ingen sådan ingång och kopplas in med Z=CALL(16384,2).
- *OLD i tolken.* Där måste kretsen startas med PRINT CALL(16384), för Z=CALL(16384) lägger Z mitt i den första raden, som OLD ska hämta tillbaka. Inkopplad går OLD direkt.
- *CHANGE* byter inte parametrarna i DEF FN.
- *Pausen.* Tangenten som avslutar en paus går inte vidare till programmet, till exempel till GET.
- *De egna orden.* Kompilatorn prövar inte gränsen på 304 byte, och ord som går förbi den skrivs över av stackarna.
- *STEPF* ligger i de sista osynliga byten i bildminnet, $7F7E–$7F7F. Ett program som använder dem krockar med kretsen.

– *START* behåller variablerna bara så länge programmet inte är ändrat, för ROM:en nollställer dem när en rad läggs in eller tas bort.
– *ED* ger ROM:en raden fram till markören. Det som står efter måste tas med högerpilen, som med mallen i ROM:ens eget `ED`.

5.8 Slutsatserna

Det går. En Smartaid i Forth får plats på `$4000`–`$5FFF`, med samma sätt att ta över tangentbordet, en skärmeditor och stegning. Den är dessutom öppen, för kommandona är ord i en tolk där användaren kan lägga till egna. Vägen som band 2 pekar på fanns alltså.

Priset är plats och tid. Tolken tar nästan en tredjedel av kretsen, och det som Smartaid 3 rymmer utöver Forthaid, fyra kommandon och skrivaren, får inte plats. Där arbetet görs av slingor i Forth över programmets byte var Forthaid först 10–15 gånger långsammare. Där ROM:en gör arbetet, som i `REN`, är skillnaden liten.

Botemedlet var att skriva de heta slingorna som maskinkodsord. Det var också vad uppskattningen före bygget räknade med. Tabellen med en bit per variabel och `SETVARS` gjorde `VAR` omkring tio gånger snabbare, `SEARCH` gjorde `FIND` 2,8 gånger snabbare och `MATCHVAR` gjorde `CHANGE` 25 % snabbare. Alla blev dessutom kortare, eftersom ett maskinkodsord tar mindre plats än de Forth-ord det ersätter. På 12 kilobyte är Forthaid nu 1,2–1,3 gånger långsammare än Smartaid 3, utom med `FIND`. Ett Forth-verktyg med sina tyngsta slingor i maskinkod kommer alltså nära ett som är skrivet helt i assembler.

Uppskattningen var sämst för `FIND`, som skulle ta 0,5–1 s över 8–10 kilobyte. Den räknade med en sökning byte för byte i programmet, men `FIND` söker i raden som den listas och gör om varje rad till text först.

5.9 Inte prövat

Kretsen är bara prövad i emulatorn, inte i en riktig ABC80. Smartaid 3 är också mätt i emulatorn, inte i en riktig maskin. Forthaid går i 9913 och 11273; en annan ROM-version känns inte igen. CMOS-tolken är prövad bara med emulatorns CMOS-minne.

Koden ligger i `utanfor/forthaid` i kodförrådet. `gor.sh` bygger kretsen ur kopior av bokens källor, och `gor.sh -p` kör de 25 proven i båda ROM-versionerna. `matning.sh` gör mätningen och räknar storleken. `MANUAL.md` beskriver kommandona, editorns tangenter, stegningen och hur man skriver egna ord. På webbsidan finns kretsen bland kretsarna, med `?krets=forthaid`, och tolken med CMOS-minnet med `?krets=cmos`.

Forthaid II

Utanför banden. Forthaid II är Forthaid från förra kapitlet, flyttad så att den går med TKN80, MyAB:s kort för 80 tecken per rad, och delad på en fast del och fem banker. Bankkortet har aldrig funnits. Det är påhittat för försöket och finns bara i emulatorn och på webbsidan. Forthaids och bokens källor ändras inte: byggskriptet kopierar dem och ändrar kopiorna. Allt är prövat i emulatorn, i ROM-versionerna 9913 och 11273, med och utan TKN80.

Forthaid fyller `$4000`–`$5FFF`. Det gör att den inte kan sitta i samma maskin som TKN80. Kapitlet prövar två vägar. Den första är att krympa Forthaid tills den ryms bredvid kortet: vad måste då bort? Den andra är att ge kretsen mer plats genom bankning, som Super Smartaid fick: hur gör man det i en trådad tolk, och vad kostar ett anrop som går över till en annan bank? Genomgången i bilagan Forth på ABC80 går lika bra med Forthaid II.

6.1 Varför Forthaid krockar med TKN80

TKN80 har sitt bildminne på `$5800`–`$5FFF`, och den övre halvan är samma minne som ABC80:s bildminne på `$7C00`–`$7FFF` (band 2, kapitlet Skärmen, avsnittet 80 tecken per rad). Forthaid krockar med kortet på tre sätt:

- *Kretsen.* Dess sista 2 kilobyte ligger på samma adresser som kortets bildminne. I emulatorn syns Forthaids kod som tecken på skärmen.
- *Vektortabellen* KTAB ligger på `$5F34`, alltså också i kortets minne.
- *Variablerna* ligger i bildminnets osynliga byte, `$7C78`–`$7F7F`, som i exempel 7 i band 2. I 80-teckensläget hamnar `$7C78`–`$7C7F`, `$7D78`–`$7D7F` och så vidare mitt i raderna 12–15 och skrivs över av utskrift och rullning. Bara `$7xF8`–`$7xFF` är osynliga i båda lägena (figuren över TKN80:s rader i samma avsnitt).

Variablerna flyttas därför i båda vägarna. Elva av dem får gömställena `$7CF8`–`$7CFF`, `$7DF8`–`$7DFF` och `$7EF8`–`$7EFD`, och stegningens flagga `$7EFE`–`$7EFF`. `$7FF8`–`$7FFF` lämnas åt exempel 1. Det är tolkens egna variabler, som `STATE`, `DP` och `LAST`, som hamnar i bildminnet, för de ska överleva mellan kommandona, och dit når inte ROM:en när den läser eller skriver en fil. Resten, som bara används medan ett kommando körs, ligger efter radbufferten i kassettbufferten, `$FB50`–`$FB6F`. De egna orden börjar därför på `$FB70` och har 272 byte i stället för 304.

6.2 Utan banker

Under `$5800` finns 6 144 byte, och Forthaid använder 8 101. Omkring 2 kilobyte, en fjärdedel av kretsen, måste alltså bort. Byggskriptet kan ta bort ett urval kommandon och de av Forthaids ord som bara de använder. Vektortabellen flyttas till `$5734` (I = `$57`).

Tabellen visar några urval och hur många byte som då blir lediga före vektortabellen:

Bort	Lediga
PEEK VAR REN OLD	−475
PEEK VAR REN CHANGE	−180
PEEK VAR REN OLD AUTO	−255
PEEK VAR OLD CHANGE	−303
PEEK REN OLD CHANGE	−690
VAR REN OLD CHANGE	2
PEEK VAR REN OLD CHANGE	175
PEEK VAR REN OLD CHANGE AUTO	395

VAR och CHANGE delar det mesta av sin kod, tabellen över variablerna, MATCHVAR och SETVARS, så det räcker inte att ta bort det ena av dem. Det minsta urval som ryms är VAR, CHANGE, REN och OLD, med 19 hjälpord. Då är två byte lediga. Kvar är LIST, FIND, PEEK, SYS, DEL, AUTO, ED, START och HELP, inkopplingen med skärmeditorn och stegningen.

Forthaid kan alltså gå med TKN80 utan bankning, men bara utan kommandona för variabler och radnummer. Det är de kommandona som en Smartaid-användare troligen hade saknat mest. Den här versionen klarar Forthaids prov i båda ROM-versionerna och med TKN80 i båda lägena, men inte rader längre än 40 tecken i 80-teckensläget. För rättelsen av det finns ingen plats.

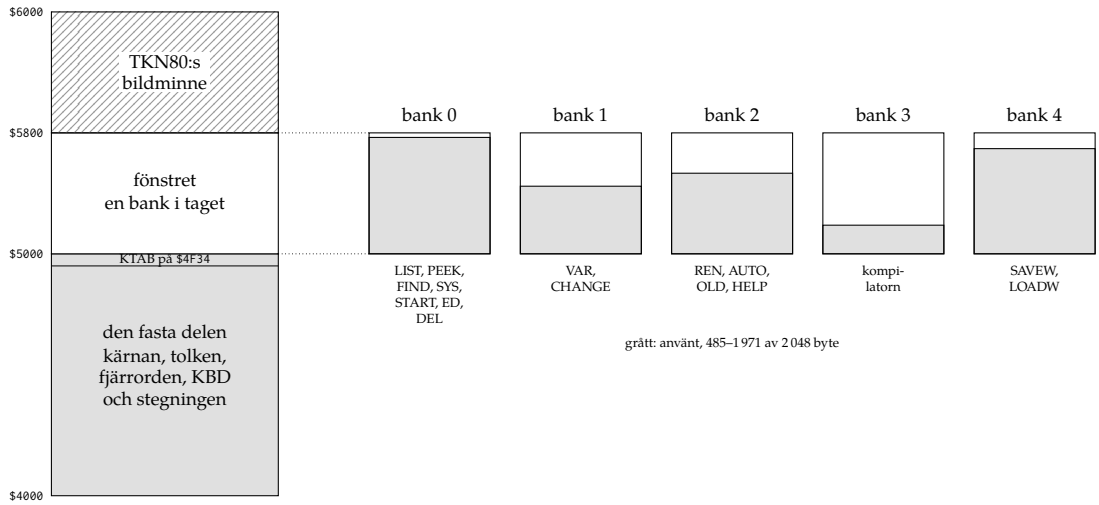
6.3 Bankkortet

Kortet med bankerna är påhittat. Det är det kort som emulatorns flagga `-b` redan hade för exempel 4 i band 2 (kapitlet Egna exempel, avsnittet Exempel 4: två banker på samma adresser):

- upp till åtta banker, här 2 kilobyte var, i ett fönster på `$5000`–`$57FF`;
- en skrivning till `$5040`+*n* kopplar in bank *n*, vilket värde som än skrivs, och en skrivning där bank *n* saknas gör ingenting;
- fönstret är skrivskyddat som en krets, och bank 0 är inkopplad vid start.

Sättet att välja bank är Super Smartaids, flyttat till fönstrets adress (band 2, kapitlet Smartaid, Superbasic och ABC80L, avsnittet Super Smartaid). Det är valt för att emulatorn och webbsidan redan kunde det, inte för att efterlikna Smartaid. Eftersom kortet bara lyssnar på skrivningar kan adresserna `$5040`–`$5047` ändå hålla kod i varje bank. Kortet har inget eget RAM, och det går inte att läsa av vilken bank som är inkopplad. Fönstret slutar under `$5800`, så TKN80:s bildminne berörs inte.

Den fasta delen ligger på `$4000`–`$4FEF` med vektortabellen på `$4F34` (I = `$4F`). Figuren visar hela området. Varje bank är fylld så långt som den används.



Forthaid II på `$4000`–`$5FFF`*, med högre adresser överst. Den fasta delen har tolken och allt som flera banker behöver, och i fönstret på* `$5000` *syns en av de fem bankerna i taget. TKN80:s bildminne på* `$5800`–`$5FFF` *lämnas fritt.*

6.4 Fjärrorden

I en trådad tolk är ett ord en lista med adresser till andra ord. En adress i `$5000`–`$57FF` betyder olika saker beroende på vilken bank som är inkopplad, så ett ord i en bank kan inte anropas med sin adress från den fasta delen eller från en annan bank. Det anropas i stället genom ett *fjärrord* i den fasta delen. Det är fem byte, ett `CALL` och två byte med banken och platsen i bankens tabell. `VAR` ligger i bank 1:

```
4C0F  CD 9E 45     F_VAR:  CALL DOFAR
4C12  01 02                DEFB 1,2              bank 1, plats 2
```

Varje bank börjar med sitt nummer på `$5000`, antalet ord i tabellen på `$5001` och från `$5002` adresserna till de ord som anropas utifrån. Kortet går inte att läsa av, så det är numret i banken som säger vilken bank som är inkopplad. Tack vare tabellen behöver den fasta delen ingen adress ur bankerna, och en bank kan byggas om utan att den fasta delen ändras.

DOFAR gör som DOCOL men lägger först undan mer. Den lägger IP och den inkopplade banken på returstacken, som har IX som stackpekare. Sedan kopplar den in den nya banken och hoppar till ordet med DE = FARX:

```
45B6  E1                POP  HL             fjärrordets två byte
45B7  7E                LD   A,(HL)         den nya banken
45B8  23                INC  HL
45B9  5E                LD   E,(HL)         platsen i tabellen
45BA  F6 40             OR   $40
45BC  6F                LD   L,A
45BD  26 50             LD   H,BANKH        HL = $5040+n
45BF  77                LD   (HL),A         koppla in den
45C0  6B                LD   L,E
45C1  5E                LD   E,(HL)
45C2  23                INC  HL
45C3  56                LD   D,(HL)
45C4  EB                EX   DE,HL          HL = ordet i banken
45C5  11 C9 45          LD   DE,FARX        IP när ordet är klart
45C8  E9                JP   (HL)
45C9  CB 45       FARX: DEFW FAREXIT
45CB  DD 7E 00    FAREXIT: LD  A,(IX+0)     den gamla banken
45CE  DD 23             INC  IX
45D0  DD 23             INC  IX
45D2  F6 40             OR   $40
45D4  6F                LD   L,A
45D5  26 50             LD   H,BANKH
45D7  77                LD   (HL),A         tillbaka
45D8  C3 69 40          JP   EXIT            till det anropande ordet
```

Ordet i banken börjar med sitt eget DOCOL, som lägger FARX på returstacken som återhopp. Returstacken ser då ut så här, med högre adresser överst:

```
IP i det anropande ordet
den bank som var inkopplad
FARX                      vid IX
```

När ordet är klart ger dess EXIT IP = FARX, och NEXT kör då FAREXIT, eftersom FARX är ett ord vars kod är FAREXIT. Den kopplar tillbaka den gamla banken och gör EXIT till det anropande ordet. Ordet i banken vet alltså inte att det anropas utifrån. Ett anrop inom banken och ett anrop av den fasta delen är vanliga adresser i tråden, som förut. DOFAR, FARX och FAREXIT är 61 byte.

Ett ord i maskinkod kan också ligga i en bank. Fjärrordet hoppar då till koden med DE = FARX, och ordets avslutande JP NEXT kör FAREXIT på samma sätt.

6.4.1 Vad ett anrop kostar

Ett prov med exempel 7:s tolk och två små banker mäter 10 000 anrop av ett tomt ord med klockan och drar ifrån en tom slinga. En klocktick motsvarar 6 T per anrop:

Anropet	T per anrop
i den fasta delen	335
inom en bank	335
från bank 1 till bank 2	754
från den fasta delen till bank 1	760

Räknat i koden kostar ett fjärrord 423 T mer än ett vanligt anrop: CALL DOFAR 17, DOFAR 226, NEXT till FARX 38, FAREXIT 74 och det extra EXIT 68. Mätningen ger 419–425. Ett anrop över gränsen tar alltså drygt dubbelt så lång tid som ett vanligt, omkring 250 µs mot 112 µs. Ett anrop inom banken kostar ingenting extra. Returstacken får tre celler per anrop över gränsen i stället för en.

Två saker går inte över gränsen. Data i en bank finns bara när banken är inkopplad, så en text från ." i en bank kan skickas till TYPE i den fasta delen men inte till ett ord i en annan bank. Ett ord som tar bort det anropande ordets återhopp med R> DROP för att lämna två nivåer fungerar inte heller, eftersom banken och FARX ligger emellan. Forthaid gör inte så.

6.4.2 Korskompilatorn

Forthaid byggs med bokens korskompilator för exempel 7. För bankerna finns en ny, som läser in den gamla och använder dess läsare och kompilator som de är. Bara bygget är nytt. En fil kan anges med en bank, `fil.fs@1`, och en karta säger vilken bank varje ord hamnar i:

```
1: PEEKC SKIPBL
1: ADV1 VARNAME VKEY MATCHVAR USES? .VAR SETVARS ALLVARS VAR
1: NAME? RENAME CHANGE
```

Ord som inte står i kartan hamnar i den fasta delen. Ett ord får ett fjärrord om det anropas från den fasta delen eller från en annan bank, används med ['] eller är en ingång, och ['] ger alltid fjärrordet, så att EXECUTE fungerar var det än görs. Ett ord med fjärrord får också sitt huvud i den fasta delen, och så gör orden på en rad h: i kartan, så att de kan skrivas vid OK. Övriga ord i bankerna syns inte för tolken. Ett litet hjälpord som PEEKC eller SKIPBL kan stå i flera banker och får då en kopia i var och en. Det kostar några byte men sparar ett fjärranrop varje gång det används inom banken.

6.5 Kommandona i bankerna

Med bankerna är alla Forthaids kommandon med igen. Den fasta delen har kärnan, tolken, fjärrorden, inkopplingen och stegningen, och de av Forthaids ord som flera banker eller inkopplingen behöver. Kommandona ligger i tre banker, och två till kom senare:

Bank	Vad	Byte	Lediga
0	LIST, PEEK, FIND, SYS, START, ED, DEL	1 971	77
1	VAR, CHANGE	1 145	903
2	REN, AUTO, OLD, HELP	1 365	683
3	kompilatorn	485	1 563
4	SAVEW, LOADW	1 781	267

Fördelningen följer kommandona. VAR och CHANGE delar sin kod och ligger tillsammans, och REN, AUTO och OLD arbetar med radnumren. De stora maskinkodsorden DETOK, MATCHVAR, SEARCH, SETVARS, (INS) och EDIT används bara av kommandona och ligger i sina banker. Korskompilatorn prövar att ingen kod i den fasta delen eller i en annan bank hoppar direkt till dem. Det som är skrivet i Forth ändras inte. Kartan bestämmer bara var orden hamnar.

Forthaids prov går i 9913 och 11273, utan TKN80 och med TKN80 i båda lägena.

6.6 80 kolumner

Med TKN80 i 80-teckensläget och en programrad längre än 40 tecken gick ED fel. Forthaids editor och LIST räknar med 40 tecken per skärmrad på fyra ställen: radläsningen byter rad vid kolumn 40, CTRL-S går inte förbi kolumn 39, CTRL-D och CTRL-P flyttar 39 minus kolumnen tecken, och LISTLINE räknar skärmraderna som radens längd delad med 40. ED trodde därför att raden tog två skärmrader, flyttade markören två rader upp och läste in fel rad, så att BASIC svarade med fel 31.

ROM:ens rutiner som editorn använder ligger i de två kilobyte som TKN80 byter ut och klarar sig själva. Det som behövs är radlängden, och den syns i radtabellen: `PEEK(885)` är `$58` i 80-teckensläget och annars `$7C` (band 2, kapitlet Skärmen). En rutin i maskinkod och ett Forth-ord `COLS` läser den och ger 40 eller 80, och de fyra ställena räknar med den. Läget läses varje gång, så det går att byta läge mellan två kommandon. `COLS` ligger i bank 0 med `LISTLINE`, och bank 0 blev 49 byte större. Utan banker finns ingen plats för det.

6.7 Tiderna

Kommandona mäts som i förra kapitlet, på samma tre program och med samma kommandon. Tiden är den emulerade maskinens, i sekunder, utöver ett program med två rader:

Programmet	Kretsen	FIND	VAR	VAR Z9	REN	CHANGE
2 060 byte	Forthaid II	0,35	0,60	0,18	0,41	0,18
	med TKN80	0,36	0,61	0,20	0,43	0,19
	Forthaid	0,35	0,60	0,18	0,41	0,18
8 180 byte	Forthaid II	1,50	1,05	0,74	2,21	0,72
	med TKN80	1,51	1,06	0,76	2,22	0,74
	Forthaid	1,50	1,05	0,74	2,21	0,72
12 266 byte	Forthaid II	2,29	1,36	1,12	3,90	1,09
	med TKN80	2,29	1,37	1,14	3,91	1,10
	Forthaid	2,29	1,36	1,12	3,90	1,09

Forthaid II tar lika lång tid som Forthaid, på mätningens tio millisekunder när. Jämförelsen med Smartaid 3 i förra kapitlet står alltså kvar. Med TKN80 i 80-teckensläget tar kommandona 0–20 ms längre, lika mycket oavsett programmets längd. Det är utskriften genom TKN80:s rutiner, inte bankningen.

Att bankningen inte syns beror på var gränsen går. Mätningen räknar hur många gånger `DOFAR` körs medan kommandot arbetar. `FIND`, `VAR`, `REN` och `CHANGE` går över gränsen en gång, när kommandot anropas, oavsett programmets längd. Varje kommando ligger med sina hjälpord i en bank, och slingorna över raderna stannar i banken eller anropar den fasta delen. Bara `VAR Z9` går fyra gånger: en gång till `VAR`, en gång till `PAGE` och två gånger för raden med Z9, till `LISTLINE` och `MORE?` i bank 0. Med 423 T per anrop är det 0,14 ms för de flesta kommandona, och en variabel som står på 40 rader skulle kosta omkring 12 ms. Hade `LISTLINE` eller `DETOK` legat i en annan bank än `FIND`, hade i stället varje rad i programmet kostat ett anrop till.

6.8 Kompilatorn i en egen bank

Den fasta delen hade 107 byte lediga före vektortabellen. För att få plats med mer flyttades tolkens kompilator till bank 3. Det är de ord som bara behövs när man definierar nya ord: `S,`, `HEADER`, `REVEAL`, `IMMEDIATE`, `:`, `;`, `VARIABLE`, `CONSTANT`, `'`, `LITERAL`, kontrollorden från `IF` till `LOOP` och `."`. Det krävde två rader i kartan, och inget annat ändrades.

Den yttre tolken når orden genom deras fjärrord, också de som är `IMMEDIATE`. Biten står i huvudet, och huvudet ligger i den fasta delen. Det som orden lägger i ett nytt ord, som `0BRANCH`, `(DO)`, `TYPE` och `EXIT`, är adresser i den fasta delen, så ett eget ord behöver aldrig bank 3 när det körs. Ett ord som kompileras gör ett anrop över gränsen för varje `:`, `;`, `IF` och så vidare, omkring 420 T extra var. Det märks inte vid tangentbordet. Den fasta delen fick 443 byte lediga i stället för 107, och Forthaid II blev 149 byte större: fjärrorden och bankens tabell.

6.9 Orden på skivan

I Forthaid försvinner de egna orden när en fil läses eller skrivs, eftersom de ligger i kassettbufferten. Med platsen i bankerna fick Forthaid II två kommandon till. SAVEW *namn* skriver de egna orden till en fil, och LOADW *namn* läser dem tillbaka, också i en annan körning. Kommandona ligger i bank 4 och finns i vektortabellen, så de går också direkt i BASIC. Skärmen är ur provet, med en diskett i ABC-DOS:

```
FORTH
: SQ DUP * ; : Q4 SQ SQ ; OK
: P 0< IF ." N" ELSE ." P" THEN ; OK
VARIABLE V 7 V ! OK
SAVEW LIB OK
SAVEW ?
OK
LOADW ?
OK
LOADW PRG.BAC ?
OK
BYE
ABC80
LOADW LIB
ABC80
SYS
BOFA C000 EOFA C000 HEAP C00A
PROGRAM 1 VAR 9 FRITT 13490
ORDEN FB70 FRITT 272
ABC80
```

Utan namn svarar SAVEW och LOADW ?, och så gör LOADW för en fil som inte är en ordfil, här ett BASIC-program som sparades före provet. Efter LOADW LIB är de egna orden de laddade, och de 272 byten på $FB70 är lediga för nya.

Filen har ett huvud på 8 byte: FW, LATEST, längden och det sista ordets huvud räknat från filens början. Sedan kommer orden, där varje adress till ett eget ord är räknad från filens början, och sist en bitkarta med en bit per byte som säger var en sådan adress börjar. LATEST är adressen till den fasta delens sista ord. En fil från en annan Forthaid II, där den fasta delens ord ligger på andra platser, ger därför ? i stället för att laddas.

SAVEW går igenom orden huvud för huvud. Länken flyttas, och i ett kolonord varje cell som pekar in bland de egna orden. Talet efter LIT och texten efter (S") hoppas över. Allt annat i ett kolonord är ett ord eller ett hopp, och hoppen efter BRANCH, 0BRANCH och (LOOP) pekar in i samma ord och flyttas på samma sätt. En VARIABLE eller CONSTANT sparas som den är, så en konstant som håller adressen till ett eget ord pekar fel efter LOADW.

LOADW läser filen till BASIC:s minne under STKTOP och sänker STKTOP lika mycket. BASIC tar sin stack därifrån vid nästa kommando. Den första LOADW sparar det gamla värdet, och en ny LOADW tar samma minne, så BASIC förlorar bara så mycket som den största filen behöver. Orden laddas alltså inte i en bank. Kortet behöver inget RAM, och de laddade orden ligger kvar när nya ord definieras på $FB70.

Filerna läses och skrivs med ROM:ens OPEN, CLOSE, DVGET och DVPUT, med en egen stack i BASIC:s lediga minne, för DOS:en använder mer stack än vad som ryms under Forths stackar. En enhet vars buffert ligger i kassettbufferten vägras, för där ligger de egna orden. Med kassetten blir det alltså ?, och SAVEW och LOADW kräver ett skivminne. En fil som inte finns ger fel 21 i BASIC, som när OPEN misslyckas annars.

Exempeldisketten har en ordfil, VERKTYG.FTH, med fyra ord efter kommandon i Smartaid-kretsarna och deras like: CROSS listar de rader som hoppar till en viss rad, RAM visar en rad byte för byte, FIX ger ett hopp till en rad som inte finns numret på nästa rad som finns, och PAC tar bort REM-raderna och gör sedan FIX. Orden tillsammans är större än 272 byte, så de skrivs in i omgångar med SAVEW och LOADW emellan. Det är just det som LOADW gör möjligt: de egna orden är inte längre begränsade till kassettbufferten.

6.10 Storleken

Forthaid II är nu 10 414 byte:

Delen	Byte	Lediga
den fasta delen	3 667	413
bank 0	1 971	77
bank 1	1 145	903
bank 2	1 365	683
bank 3	485	1 563
bank 4	1 781	267
Forthaid II	**10 414**	**3 906**
Forthaid	8 101	75

Den fasta delen räknas utan de lediga byten före vektortabellen. Därtill finns 16 lediga byte sist i den fasta delen, som i Forthaid. Med samma kommandon som Forthaid, före kompilatorn och skivan, var Forthaid II 8 436 byte, 335 mer än Forthaid. Fjärrorden var 22 à 5 byte i den fasta delen och 2 i bankens tabell, DOFAR, FARX och FAREXIT 61 och bankernas nummer och antal 6. Resten var kopiorna av små hjälpord i flera banker, COLS för 80 kolumner och huvudena för de ord som anropas utifrån. Nu finns 45 fjärrord.

6.11 Begränsningarna

– *Kortet* har inte funnits. Forthaid II går bara i emulatorn i terminalen och på webbsidan. Emulatorns SDL-fönster har inget bankkort.
– *Ingen automatisk start*, som i Forthaid. Kretsen kopplas in med Z=CALL(16384,2).
– *Smartaid 3:s* RESUME, DIR, LIB, FOR i direktläge och PR: finns inte.
– *Över bankgränsen* går inte data i en bank eller R> DROP. Ett eget ord kan inte läggas i en bank.
– *SAVEW och LOADW* kräver ett skivminne, och en fil från en annan version av Forthaid II laddas inte. Efter Z=CALL(16384) ligger BASIC:s stack under STKTOP, och LOADW kan då bara ta det minne som en tidigare LOADW redan har tagit.
– *Gömställena.* Ett program som självt använder bildminnets osynliga byte krockar med kretsen.

6.12 Slutsatserna

Forthaid kan gå med TKN80 utan bankning, men bara om kommandona för variabler och radnummer tas bort. Med ett enkelt bankkort, av samma slag som Super Smartaids, får alla kommandon plats igen, och 3 906 byte blir lediga.

Bankningen i en trådad tolk gick att lägga i tre korta rutiner, 61 byte, och fem byte per ord som anropas utifrån. Orden själva ändrades inte, och korskompilatorn bestämmer var de hamnar. Ett anrop över gränsen tar drygt dubbelt så lång tid som ett vanligt, men det märks inte i tiderna,

för gränsen går mellan kommandona och inte inne i deras slingor. Det är kartan över orden som avgör det, och den är det viktigaste valet.

Den nya platsen gick till sådant som Forthaid saknade: 80 kolumner, kompilatorn flyttad ur vägen och egna ord som sparas på skivan. Bankning fanns på riktigt i Super Smartaid och Magnum. Här visar det sig att den också passar en trådad tolk, och att en öppen krets kan växa med den.

6.13 Inte prövat

Kortet finns inte, så inget av det här är prövat i en riktig ABC80, och TKN80 bara i emulatorn. Forthaid II går i 9913 och 11273, och med en annan ROM-version tar den kanske inte över. Hur ett sådant kort hade byggts är inte undersökt.

Koden ligger i `utanfor/forthaid2` i kodförrådet. `gor.sh` bygger den fasta delen och bankerna ur kopior av Forthaids och bokens källor, `gor.sh -p` kör de 26 proven i båda ROM-versionerna, utan TKN80 och med TKN80 i båda lägena, och `gor.sh -1` bygger versionen utan banker. `prov/fil.sh` prövar `SAVEW` och `LOADW` med ABC-DOS, och `matning.sh` gör mätningen. `MANUAL.md` beskriver hur kretsen används. På webbsidan finns den med `?krets=forthaid2`, med TKN80 med `&tkn80=pa` och med en diskett med `&skiva=system`.

Ett färggrafikkort i HR-80:s anda

Utanför banden. Libers grafikkort HR-80 fanns, men det enda som är kvar av det är en annons. Kapitlet beskriver en egen tolk som gör det som annonsen säger. Det som annonsen inte säger är gissat, och det står här vad som är gissat. Tolken är inte en kopia av kortets program, och dess kod är skriven nu. Idén, ett kort med egen processor som tar emot kommandon på V24, är från tiden; Raspberry Pi Pico i slutet av kapitlet är det inte. Allt är prövat i emulatorn och på webbsidan, inget på ett riktigt kort.

ABC80 har en svartvit skärm med grafik av tecken, två gånger tre punkter per ruta. Den som ville ha färg fick köpa en enhet med eget bildminne (band 2, kapitlet Skärmen, avsnittet Färg och högupplöst grafik). Ett av de sista sådana tilläggen var HR-80. Kapitlet frågar hur ett sådant kort kan ha fungerat och prövar det: en tolk för kortet, en väg dit från ABC80 och ett program som ritar.

7.1 Annonsen

Liber annonserade HR-80 hösten 1984, med artikelnummer 10650 för expansionsenheten och 10652 i en egen låda på V24. Annonsen finns som sidan 22 i samlingen av Sala Teknikutbildnings prislistor och produktblad. Enligt den har kortet en egen processor, vilken sägs inte, 18 kilobyte RAM och 4 kilobyte EPROM. Det ger 256 x 256 punkter i fyra av åtta färger, eller 512 x 256 punkter svartvitt och 80 tecken per rad. Kommandona sägs vara desamma som för ABC 800:s grafiktillsats, med förbättringar. Manualen har inte gått att hitta, varken hos devili.iki.fi, abc80.net, i ABC-bladet, i Libris eller på nätet i övrigt.

Luxors grafikkort till ABC 800 är troligen inte den grafiktillsats som annonsen menar. Det styrs med BASIC II:s satser `FGPOINT`, `FGLINE`, `FGFILL` och `FGCTL` (band 2, kapitlet BASIC II på ABC80 och ABC 800), inte med tecken. HR-80 styrs med tecken, som en terminal. Kortet var alltså en liten grafikterminal: processorn kör ett program i EPROM:et som läser tecknen från V24, och listan i annonsen är det programmets kommandon.

Annonsen räknar upp koderna och vad de gör, men inte deras argument. Tabellen visar koderna och argumenten som tolken här tar emot; argumenten är gissade, utom för kod 18:

Kod	Annonsen	Argumenten här
1	välj färg	färgen 0–3
2	sätt punkt	x, y
4	linje	x, y, från den fasta punkten
5	fyll	x, y
8–11	markören vänster, höger, ned, upp	
12, 13	rensa skärmen, vagnretur	
14, 15	markören på, av	
16, 17	semigrafik, sammanhängande och åtskild	
18	markören	rad + 32, kolumn + 32
19	absolut adressering	y, x i punkter
20, 21	förgrund, bakgrund	färgen 0–3
22	standard	
24	rensa raden	

Kod	Annonsen	Argumenten här
25	markören relativt	rader, kolumner
28	fast punkt	x, y
29–31	punkt, linje och fyll relativt	dx, dy, −128 till 127
32–127	tecken	
128–253	kör ett makro	
ESC 48–51	färgekvivalens	en av de åtta färgerna
ESC 52, 56	40 tecken i färg, 80 svartvitt	
ESC 61	markören	rad + 32, kolumn + 32
ESC 68	definiera makro	numret, 66 eller 82, ..., 254
ESC 73, 82	initiera om, reset	

Annonsen nämner också understrykning, dubbel höjd, omvänd video och en blinkande markör, men inte med vilka koder. Koderna 3, 6, 7, 23 och 26 står inte i listan.

7.2 Det som är gissat

För att tolken ska kunna rita något måste mycket bestämmas som annonsen inte säger. Valen har gjorts så att de liknar ABC 800 där det går:

- *Argumenten* är hela byte, `CHR$(x)` i BASIC: x och y 0–255, de relativa −128 till 127. y = 0 är nederst, som i ett diagram. Bara kod 18 har + 32 i annonsen, och ESC 61 får det också, så att ingen av de två skickar styrkoder.
- *Färgerna* är ABC 800 C:s. Skärmen får R, G och B av och på, och de åtta färgerna numreras som ABC 800:s textfärger: 0 svart, 1 röd, 2 grön, 3 gul, 4 blå, 5 magenta, 6 cyan och 7 vit. ESC 48–51 sätter var och en av bildens fyra färger till en av de åtta, så att varje rad i `FGCTL`:s färgvalstabell går att få. Från början är de svart, röd, grön och vit, som `FGCTL 6`.
- *Texten* står i rutor om 6 x 10 punkter med ABC80:s teckensnitt, 40 eller 80 per rad och 24 rader. Kod 19 sätter rutan på en punkt, så att text kan stå var som helst i bilden.
- *Semigrafiken* (16 och 17) ritar teledatans mosaik, två gånger tre block per ruta, tills kod 22.
- *Fyllningen* byter den sammanhängande ytan med startpunktens färg mot ritfärgen.
- *Makrona* slutar alltid med 254. Ett makro med instruktioner (82) tolkas som om tecknen kom på V24. En bitmapp (66) börjar med bredden och har sedan sju punkter per byte, så att 254 aldrig kan förekomma i den.

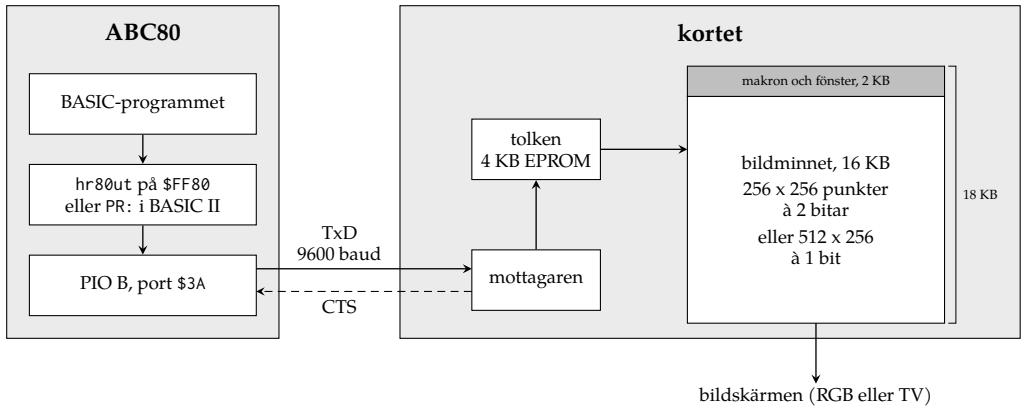
Understrykningen, dubbel höjd och omvänd video finns inte, eftersom koderna är okända. Dyker manualen eller en kopia av EPROM:et upp är det tolken som byts, inte resten.

7.3 Tolken

Tolken är skriven i C, i en fil, utan `malloc`, utan `stdio` och utan något från emulatorn. Den har tre ingångar: `hr80_starta` nollställer kortet, `hr80_tecken` tar emot ett tecken från V24, och `hr80_rita` gör om bildminnet till punkter i värdens färger. Samma fil byggs för emulatorn i terminalen, för SDL-fönstret, för webbläsaren och för en mikrokontroller.

Tolken har ett tillstånd och tar ett tecken i taget, i den här ordningen: pågår en makrodefinition sparas tecknet till 254; väntar ett kommando på argument blir tecknet ett argument, och kommandot utförs när alla har kommit; efter ESC väljer tecknet ett ESC-kommando; 128–253 kör ett makro; 32–126 ritar ett tecken; resten är styrkoder. Ett makro med instruktioner kör tecknen genom samma ingång, och ett makro får köra andra makron åtta nivåer djupt.

Bildminnet är 16 kilobyte, som på kortet. I färgläget har det 256 x 256 punkter med två bitar vardera, ett index till de fyra färgerna. I det svartvita läget har det 512 x 256 punkter med en



Vägen från BASIC-programmet till bilden. ABC80 skickar tecknen bitvis på PIO B, kortet tar emot dem och tolken ritar i bildminnet. Bildminnet, makrona och fönstertabellen är tillsammans annonsens 18 kilobyte. CTS (streckat) används inte i emulatorn, där kortet alltid är klart.

bit, och grafikens x dubblas, så att samma program ritar samma bild i båda lägena. De 2 kilobyte som blir kvar av de 18 som annonsen anger delar makrona med de påhittade fönstren (`utanfor/fonster/`): fönstertabellen och händelsekön tar 384 byte, och makrona får 1 664. Linjerna ritas med Bresenhams metod. Fyllningen går rad för rad: den fyller en sträcka åt vänster och höger och lägger sträckorna ovanför och nedanför på en egen stack.

Byggd med `clang -Os` för en ARM Cortex-M33 var tolken före fönstren 4 212 byte, och av det är teckensnittet 928. Det säger inte mycket om vad den hade blivit i en processor från 1984, men den är av samma storlek som annonsens EPROM. Minnet räcker sämre: tabellerna över makrona tar omkring 1 kilobyte, och fyllningens stack, 4 096 sträckor à 4 byte, tar 16 kilobyte. Ett kort med 18 kilobyte hade fått nöja sig med en stack på några hundra byte och lämna stora, krokiga ytor ofyllda.

Fyra provbilder prövar tolken utan maskinen: text, färger, en ram och en fyllning; semigrafiken, färgekvivalensen och de relativa kommandona; 80 tecken med två diagonaler; och två makron, ett med instruktioner och en bitmapp. Varje bild jämförs med en sparad kontrollsumma.

7.4 På V24

I emulatorn sitter kortet på PIO B, port `$3A`, där V24 finns (band 1, kapitlet ROM:en och hårdvaran, avsnittet PIO B (port `$3A`/`$3B`): V24 och kassetten). Kortets mottagare läser TxD, bit 3, med samma avkodare som emulatorns V24-skrivare: den mäter bitarna i T-cykler och ger ett helt tecken till `hr80_tecken`. Kortet är alltid klart, för tolken ritar på en gång. Först ett riktigt kort, som ritar långsammare än tecknen kommer, behöver CTS.

BASIC II skriver redan på V24 med enheten PR: (band 2, kapitlet Skrivarkretsen 9704, avsnittet Till skrivaren). Exemplet öppnar den med `OPEN "PR:VSA70A99.7" AS FILE 1`: V24, tecknen som de är, inga NUL, ingen radbrytning och 9600 baud. Varje `PRINT #1` slutar med semikolon, så att inget CR LF kommer med.

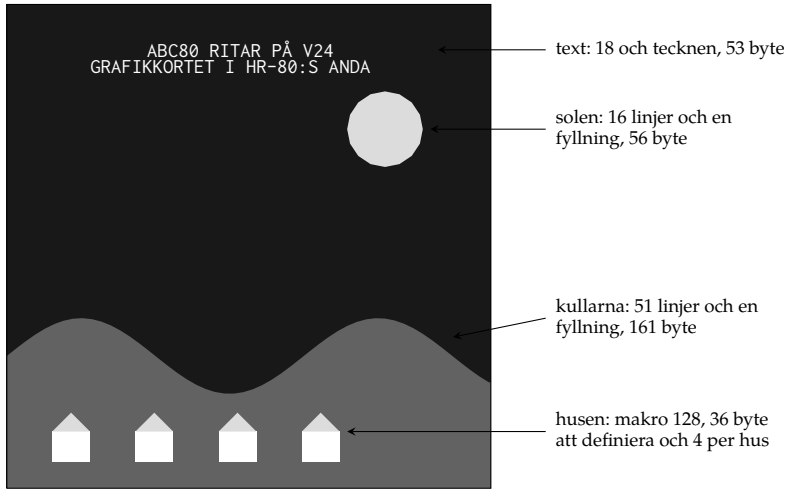
ABC80:s egen ROM har ingen V24-enhet. Exemplet lägger därför en rutin på 59 byte på `$FF80`, i de 128 byte som bruksanvisningen lämnar för egen maskinkod (band 1, kapitlet Interpretatorns arkitektur, avsnittet Minnet), och anropar den med `Z=CALL(65408,T)` för varje tecken. `EXCALL` lägger T i DE. Rutinen slutar på `$FFBA`, före UFD-DOS:ens korttabell på `$FFC0`. Den gör som `V24OUT` i skrivarkretsen: väntar på CTS, stänger av avbrotten och skickar startbiten, åtta databitar och stoppbiten. Är inget kort inkopplat väntar den, men CTRL-C går tillbaka utan att skicka, och BASIC stannar efter `CALL`. Slingan som skickar bitarna:

```
FF98  DB 3A       BIT:   IN   A,($3A)
FF9A  38 04              JR   C,ETT          biten står i CY
FF9C  E6 F7              AND  $F7            TxD = 0
FF9E  18 04              JR   UT
FFA0  F6 08       ETT:   OR   $08            TxD = 1
FFA2  00                 NOP                 två NOP: som AND och JR
FFA3  00                 NOP
FFA4  D3 3A       UT:    OUT  ($3A),A
FFA6  3E 0F              LD   A,15           vänta 235 T
FFA8  3D          DROJ:  DEC  A
FFA9  20 FD              JR   NZ,DROJ
FFAB  37                 SCF
FFAC  CB 19              RR   C              nästa bit, ettor uppifrån
FFAE  10 E8              DJNZ BIT            tio bitar
```

Det är samma knep som i skrivarkretsen: biten står i CY, `RR C` skiftar fram nästa och `SCF` fyller på med ettor, så att stoppbiten kommer av sig själv. Startbiten är CY = 0 från `AND $02` i väntan på CTS. Skillnaden är att vägarna för 0 och 1 här skiljer sig med en T-cykel i stället för sju: de två `NOP` väger upp `JR UT`. Ett varv tar 315 eller 316 T-cykler, och med ABC80:s 2,9952 MHz blir det 9 480–9 510 baud, en procent från 9600, samma som skrivarkretsen.

7.5 Exemplet

Exemplet HR80 finns på båda exempeldisketterna, för ABC80:s ROM och för BASIC II. Det ritar samma bild med båda: blå himmel, en gul sol, gröna kullar, fyra hus och två rader text. Programmet sätter först de fyra färgerna med ESC 48–51, ESC följt av siffrorna 0–3, blå, gul, grön och vit, och rensar bilden med blått. Solen är 16 linjer runt en punkt, som sedan fylls. Kullarna är 51 linjer efter en sinuskurva, fyllda under. Huset är makro 128: väggarna i vitt med fyra relativa linjer och en fyllning, och taket i gult med två linjer och en fyllning. Varje hus är sedan fyra byte, en fast punkt och makrots nummer.



Bilden från exemplet HR80, ritad efter kommandona och i gråskala som häftets andra figurer, med hur många av de 335 tecknen varje del tar. De 13 som är kvar sätter färgerna och rensar bilden.

Hela bilden är 335 tecken. Med kod 4 kostar varje linje tre byte, så kullarna, som är en kurva, tar nästan hälften. Makrot sparar mer än det kostar redan vid två hus: utan det hade de fyra husen tagit 136 byte i stället för 52.

I emulatorn har alla 335 tecken kommit fram sex sekunder efter `RUN` med ABC80:s ROM. Av det är bara 0,35 sekunder själva tecknen på ledningen: tio bitar à 1/9600 sekund gånger 335. Resten är BASIC, som bygger strängarna med `CHR$` och tar ut dem med `MID$` och `ASC`, ett `CALL` per tecken. Ett riktigt kort hade alltså haft gott om tid att rita, och CTS hade sällan behövts med det här programmet.

7.6 Kortet i emulatorn

Kortet kopplas in på tre sätt. I terminalen tar `abc80 -H bild.png` emot tecknen och skriver bilden till en fil när körningen slutar. Emulatorns SDL-fönster öppnar med `-H` ett andra fönster, HR-80, med bilden och kortets markör. På webbsidan finns kortet i panelen Grafikkortet, eller med `?v24=hr80`: bilden kommer fram på en lapp vid de första tecknen, och den kan sparas som PNG.

7.7 Ett riktigt kort

Tolken är skriven så att den kan bli programmet i ett riktigt kort. Det kortet är inte byggt, men det är tänkt så här. V24 går genom en nivåomvandlare, till exempel en MAX3232, från $\pm$12 V till 3,3 V. En Raspberry Pi Pico 2 W är kortets processor, och tolken är dess EPROM. Den ena av Pico:ns två kärnor tar emot tecknen från UART:en i en buffert och drar CTS när bufferten börjar bli full, och den andra ritar och visar bilden. Bilden går ut som DVI genom ett Pico DVI Sock, som RP2350:s HSTX klarar utan överklockning. 720 x 576 i 50 Hz ger bildens 256 rader två gånger och liknar ABC80:s PAL-bild, men alla skärmar tar inte 576 rader över DVI. Minnet räcker gott: bilden tar 32 av 520 kilobyte.

Emulatorn simulerar kortet fram till kontakten: tecknen in, CTS ut och bilden. Pico:ns egna tider simuleras inte.

7.8 Begränsningarna

- *Gissningarna.* Argumenten, färgnumren, texten och makronas form är gissade. Ett program skrivet för det riktiga kortet går troligen inte här utan ändringar.
- *Attributen* understrykning, dubbel höjd och omvänd video finns inte.
- *Expansionsenheten.* Versionen för ABC-bussen, 10650, är inte gjord, eftersom kortets nummer är okänt.
- *CTS* används inte. Kortet i emulatorn är alltid klart.
- *Fyllningen* har en större stack än ett kort från 1984 hade haft plats för.

7.9 Slutsatserna

Ett grafikkort som tar emot tecken på V24 passar ABC80 bra. Maskinen behöver ingen ny enhet i ROM:en, bara 59 byte maskinkod eller BASIC II:s PR:, och kortets egen processor gör allt arbete. Annonsens lista räcker för att skriva en tolk som ritar riktiga bilder, även om argumenten är gissade. Tolken blev utan fönstren omkring 4 kilobyte, och bild och makron ryms i 18 kilobyte, som på kortet. Det som inte ryms är en enkel fyllning; där hade det riktiga kortet fått välja något snålare.

Med 9600 baud är det inte ledningen som sätter takten, utan BASIC-programmet som bygger tecknen. Makrona är därför mest värda för att spara programrader, mer än för att spara tid på ledningen.

Nästa försök, fönster på kortet med en mus och händelser tillbaka till ABC80, står i kapitel 8. Fönstren och musen är påhittade, och de gör tolken två till två och en halv gånger större. Med dem hade kortet behövt 16 kilobyte EPROM i stället för 4, och mer arbetsminne. Emulatorns kort visar alltså hur det kunde ha varit, inte HR-80.

7.10 Inte prövat

Inget av det här är prövat mot ett riktigt HR-80 eller på en riktig ABC80, och Pico-kortet är inte byggt. Sändrutinens tider är räknade och prövade mot emulatorns avkodare, inte mot en riktig mottagare.

Koden ligger i `emulator/` i kodförrådet: tolken i `karna/hr80.c` och `hr80.h`, kopplingen till PIO B i `karna/hr80v24.c`, sändrutinen i `exempel/program/hr80ut.asm` och exemplet i `exempel/program/text/HR80.BAS` och `text_b2/HR80.BAS`. `make hr80prov` kör provbilderna. Avsnittet HR-80 i `emulator/PLAN.md` har alla gissningar och planen för kortet. På webbsidan finns kortet med `?v24=hr80`.

Fönster på ABC80

Utanför banden. Det första programmet i kapitlet är en ny version av ett demo från 80-talet; demot finns inte kvar, och det nya är skrivet efter det som jag minns. Fönstren, musen och händelserna på grafikkortet i HR-80:s anda (kapitel 7) är påhittade: annonsen säger inget om dem, och det är inte troligt att HR-80 hade något sådant. Allt är prövat i emulatorn och på webbsidan, inget på en riktig ABC80.

ABC80 har en skärm, 24 rader med 40 tecken, och BASIC skriver på den uppifrån och ned. Kapitlet prövar hur fönster hade kunnat se ut på maskinen, i två steg: först textfönster i BASIC, styrda med koder i det som skrivs ut, sedan samma koder på grafikkortet från förra kapitlet, som får fönster, en mus och ett sätt att berätta för ABC80 vad musen gjort.

8.1 Demot

På 80-talet pratade jag med Olle Wolkert om *windows* för ABC80 och ABC 800, och gjorde då ett enkelt demo i BASIC: fönster i textläget, ritade med - för vågräta streck, ! för lodräta och + i hörnen. Två saker minns jag. Programmet ritades och förklarades med Jacksons strukturerade programmering, JSP. Och fönstren styrdes med koder i texten, som man styrde en skrivare: ett ESC och en bokstav med argument mitt i det som skrevs ut. Programmet behövde då bara skriva ut, och koderna valde fönster, ritade ramen eller rensade.

FONSTER på exempeldisketten (`RUN DR0:FONSTER`) är en ny version. Den öppnar tre fönster, Anteckningar, Koderna och Inköp, och tar sedan emot tangenter:

Tangent	Gör
Tab eller högerpil	lyfter det understa fönstret, som blir aktivt
Ctrl-X	stänger det aktiva fönstret
Ctrl-A	öppnar det första stängda fönstret igen
RETURN, vänsterpil	ny rad, tar bort det sista tecknet
Ctrl-C	avbryter programmet; fönstren blir kvar

Det aktiva fönstret har = i överkanten och ligger alltid överst. Det som skrivs hamnar där, bryts vid ramen och rullar när fönstret är fullt. Högst åtta fönster går att öppna.

8.2 Koderna

Allt som programmet skriver går genom en rutin, `UT` på rad 1000, som tar emot `A$` och tolkar den tecken för tecken. ABC80 har ingen ESC-tangent, så koderna kommer bara från programmet. Från tangentbordet släpps bara vanliga tecken, RETURN och vänsterpil fram till `UT`, så att ingen tangent kan starta en kod. Talen efter bokstaven skrivs som tecken, talet + 32, som markörkoden till grafikkortet, så att de aldrig blir styrtecken:

Koden	Gör
ESC O n r k h b titel CR	öppnar fönster n på rad r, kolumn k, h rader högt och b tecken brett med ramen, med titeln
ESC V n	väljer fönster n och lyfter det överst
ESC S n	stänger fönster n
FF	rensar det aktiva fönstret
CR	ny rad
BS	tar bort det sista tecknet på raden

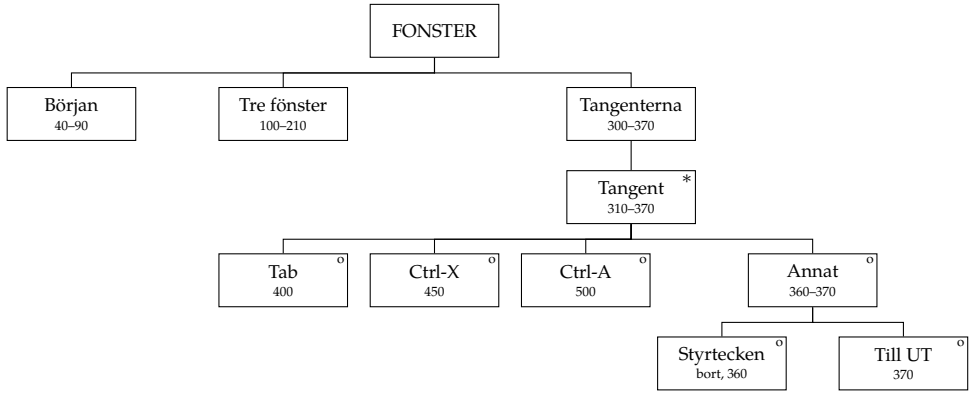
Ett fönster som öppnas igen är tomt, och ett öppet fönster som öppnas igen stängs först. Fönstret flyttas in på skärmen om det inte ryms; rad 23 är hjälpraden.

Programmet sparar inte det som ligger under ett fönster. Varje fönster har i stället sina rader i `L$`, en strängvektor med plats för 80 rader om 38 tecken, som fönstren får en bit var av när de öppnas. När ett fönster stängs suddas dess ruta, och de fönster som är kvar ritas om nerifrån och upp, men bara inom rutan. Att lyfta ett fönster kostar mindre: det ritas helt ovanpå, och det förra aktiva får bara sin överkant ritad om, med `-` i stället för `=`. Grafikkortet gör likadant, av ett annat skäl: där räcker inte minnet till att spara något.

Alla variabler är heltal, `I%` och `1%`. Med flyttal tar en slinga med `ASC(MID$(…))` omkring 4 millisekunder per varv i emulatorn, med heltal omkring en, och starten med de tre fönstren tog mer än åtta sekunder i stället för tre och en halv. Vanlig text skrivs dessutom en bit i taget, fram till ramen eller nästa styrtecken (rad 2300), och inte tecken för tecken.

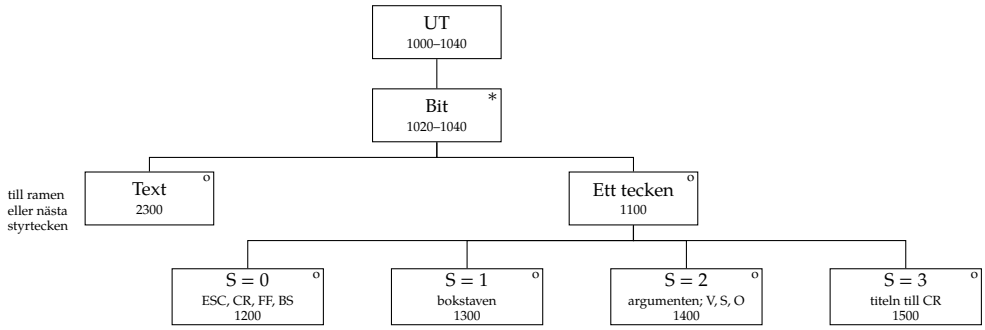
8.3 JSP

I JSP ritas programmet som ett träd av rutor. Rutorna under en ruta utan tecken är en sekvens, från vänster till höger; en stjärna betyder att rutan upprepas, och en ring att den är ett av flera val. Varje ruta är en del av programmet, och radnumren står i rutan. Huvudprogrammet är en sekvens med en upprepning sist:



Huvudprogrammet i FONSTER i JSP. En stjärna är upprepning, en ring val. Styrtecken från tangentbordet, utom RETURN och vänsterpil, tas bort.

Rutinen `UT` har ett tillstånd, `S%`, som säger vad nästa tecken är: text eller styrtecken (0), bokstaven efter ESC (1), ett argument (2) eller ett tecken i titeln (3). Valet efter tillståndet är samma sak som en skrivare gör med sina koder, och samma sak som grafikkortets tolk gör:



Rutinen UT *i JSP. Rutinerna som öppnar, väljer och stänger* (2000–2200) *använder samma tre delar: 2700 tar bort ett fönster ur ordningen, 2800 lägger det överst och ritar det, och 3000 ritar ett fönster inom en rektangel.*

8.4 Fönstren på kortet

Grafikkortets tolk har fått samma koder, i ESC-koder som annonsen inte använder, och två till. Talen är + 32, som ovan:

Koden	**Gör**
ESC O n r k h b titel CR	öppnar fönster n, 0–7, på textrad r, kolumn k, h rader högt och b tecken brett med ramen; det blir överst och valt
ESC V n	väljer fönster n och lyfter det överst
ESC T n	väljer fönster n utan att lyfta det; 8 är skrivbordet, ytan utanför fönstren
ESC S n	stänger fönster n
ESC ?	frågar efter en händelse; svaret är fyra byte på V24

Kortet ritar ramen med en titelrad och en stängningsruta, och det översta fönstret har titelraden fylld. Efter `V` eller `T` gäller annonsens kommandon i det valda fönstret: grafikens $(0, 0)$ är fönstrets nedre vänstra hörn innanför ramen, markören räknas innanför ramen, FF rensar bara fönstret, texten bryts och rullar där, och allt kapas vid ramen och vid fönstren ovanför. Ett program som ritar på hela bilden ritar alltså i ett fönster utan ändringar.

Inget sparas under fönstren, eftersom bilden tar 16 av kortets 18 kilobyte. När ett fönster stängs, lyfts eller flyttas får det som blev synligt bakgrundsfärgen, och kortet märker det fönster som tappade innehåll. ABC80 får sedan veta att fönstret ska ritas om och ritar det igen, som när ett fönster i X Window System får *expose*.

8.5 Musen

Musen sitter på kortet, inte på ABC80, som en styrspak med en knapp: ABC80:s enda V24 går redan till kortet. Kortet ritar pekaren ovanpå bilden, inte i bildminnet, så att inget behöver sparas under den. Fönstren sköter kortet själv:

– ett tryck i stängningsrutan stänger fönstret;
– ett tryck i titelraden lyfter fönstret, och medan knappen hålls dras en streckad ram; där knappen släpps flyttas fönstret, med innehållet, i hela textrutor;
– ett tryck i fönstret lyfter det och ger ett klick, på skrivbordet bara klicket.

Det valda fönstret byts inte när musen lyfter ett annat, och ABC80 märker inget förrän det frågar.

8.6 Händelserna

ABC80 läser V24 bit för bit i programvaran: RxD är bit 0 på PIO B, port `$3A`, och varje bit måste läsas när den kommer. Ett tecken som kommer medan BASIC gör annat går förlorat. Därför sänder kortet aldrig på eget initiativ. Händelserna väntar i en kö på kortet, som rymmer 16, och ABC80 frågar med `ESC ?`. En millisekund efter frågan sänder kortet svaret, alltid fyra byte, i 9600 baud:

Svaret	Betyder
K n x y	klick i fönster n (8 skrivbordet), grafikens x och y i fönstret
L n 0 0	fönster n lyftes
F n r k	fönster n flyttades till textrad r, kolumn k
S n 0 0	fönster n stängdes
R n 0 0	fönster n (8 skrivbordet) ska ritas om; kommer när kön är tom
0 0 0 0	inget

`R` ligger inte i kön. Kortet har en bit per fönster som tappat innehåll och svarar med `R` för ett av dem i taget, först när kön är tom.

Rutinen som frågar, `hr80in`, är 64 byte på `$FFBB`, direkt efter sändrutinen `hr80ut` från förra kapitlet, och anropas med `Z=CALL(65467)`. Den skickar ESC och ? med `hr80ut`, stänger av avbrotten och väntar på startbiten i en slinga om 48 T-cykler. Kommer ingen på omkring en sekund, 65 536 varv, slutar den; den första byten är då 0, och BASIC ser *inget*. Varje bit läses mitt i: först en och en halv bit efter startbitens flank, sedan var 310:e T-cykel. Slingan som läser en byte:

```
FFDC  3E 07       FUNNEN: LD   A,7            en halv bit till
FFDE  3D          DROJ1:  DEC  A
FFDF  20 FD               JR   NZ,DROJ1
FFE1  06 08               LD   B,8            åtta bitar
FFE3  3E 11       BIT:    LD   A,17           vänta 17 x 16 − 5 = 267 T
FFE5  3D          DROJ2:  DEC  A
FFE6  20 FD               JR   NZ,DROJ2
FFE8  DB 3A               IN   A,($3A)        11 T
FFEA  1F                  RRA                 RxD till CY
FFEB  CB 19               RR   C              lägsta biten först
FFED  10 F4               DJNZ BIT            7 + 267 + 11 + 4 + 8 + 13 = 310 T
```

9600 baud är 312 T-cykler vid ABC80:s 2,9952 MHz, så slingan går 0,6 procent för fort, och den åttonde biten läses 14 T tidigare i biten än den första, väl inom biten. Mellan byten väntar rutinen ut stoppbiten och letar sedan upp nästa startbit. Avbrotten är av, men klockans NMI går inte att stänga av. Den tar omkring 110 T; kommer den mitt i en byte flyttas de bitar som återstår så mycket, men de läses fortfarande inom sin bit. Svaret hamnar på `$FFFC`–`$FFFF`, och BASIC läser det med `PEEK(65532)` till `PEEK(65535)`.

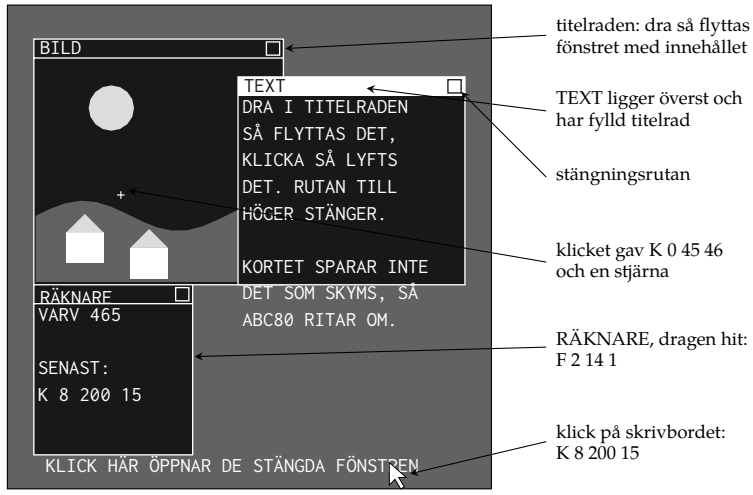
8.7 Exemplet

FONSTER2, på exempeldisketten (`RUN DR0:FONSTER2`) och som band, lägger `hr80ut` och `hr80in` på `$FF80`–`$FFFA` från `DATA`-satser och öppnar tre fönster på kortet: BILD med bilden från förra kapitlet, TEXT och RÄKNARE. Sedan frågar programmet kortet hela tiden:

```
310 Z=CALL(65467) : S%=PEEK(65532) : N%=PEEK(65533)
320 X%=PEEK(65534) : Y%=PEEK(65535) : IF S%=0% THEN GOSUB 600 : GOTO 310
330 H$=CHR$(S%)+NUM$(N%)+NUM$(X%)+NUM$(Y%) : PRINT H$
340 IF S%=82% AND N%=8% THEN GOSUB 380
345 IF S%=82% AND N%<3% THEN ON N%+1% GOSUB 500,550,580
350 IF S%=75% THEN GOSUB 400
360 IF S%=70% AND N%<3% THEN R%(N%)=X% : K%(N%)=Y%
370 IF S%=83% AND N%<3% THEN A%(N%)=0%
375 GOSUB 620 : GOTO 310
```

Inget svar räknar upp RÄKNARE (rad 600). `R` (82) ritar fönstret igen: skrivbordets rad längst ned, bilden, texten eller räknarens rubriker. Ett klick, `K` (75), i BILD lägger en stjärna där, och ett klick på skrivbordet öppnar de stängda fönstren igen, där de senast var; `F` (70) sparar det nya läget och `S` (83) att fönstret är stängt. Varje händelse skrivs också på ABC80:s egen skärm.

Med ABC80:s ROM är de tre fönstren ritade omkring sex sekunder efter `RUN`, och som i förra kapitlet är det BASIC som tar tiden, inte ledningen.



Kortets bild med FONSTER2 efter musen i provet: BILD är klickad, RÄKNARE dragen ned till vänster, och TEXT stängd och öppnad igen med ett klick på skrivbordet. Ritad efter emulatorns bild, med svaren som ABC80 fick.

Provet kör FONSTER2 i emulatorn med musen ur en fil. Klicket i BILD, som låg under TEXT, ger `L 0`, `K 0 45 46` och, när kön är tom, `R 0`, eftersom den del av BILD som låg under TEXT är tom. Dragningen av RÄKNARE ger `L 2` och `F 2 14 1` och sedan `R 1` och `R 8` för det som RÄKNARE lämnade. Stängningen av TEXT ger `S 1`, `R 0` och `R 8`, och klicket på skrivbordet `K 8 200 15`, varpå programmet öppnar TEXT igen.

8.8 I emulatorn

Musen finns i alla tre värdarna. På webbsidan är musen på kortets bild kortets mus, i panelen Grafikkortet eller med `?v24=hr80`; en dragning fortsätter utanför bilden, och ABC80:s skärm behåller tangentbordet. I SDL-fönstret, `abc80f -H`, är datorns mus i HR-80-fönstret kortets mus, och datorns pekare göms där, eftersom kortet ritar sin egen. I terminalen läser `abc80 -H bild.png -Hm fil` musen ur en fil, en rad `ms dx dy knapp` per rörelse, räknat från det att tangenterna skrivits; så körs provet ovan.

8.9 Storleken

Hade fönstren fått plats på kortet? Ingen kompilator ger det som skrevs för hand 1984, men tolken före fönstren är en kontroll: den gör ungefär det som annonsen säger, och HR-80:s tolk fick plats i 4 kilobyte EPROM. Tolken kompilerad för fyra processorer, i byte kod och konstanter med teckensnittet:

Processor	Före fönstren	Med fönster och mus	Gånger
Z80, SDCC	7 903	19 694	2,5
AVR, clang `-Oz`	6 568	15 677	2,4
MSP430, clang `-Oz`	4 370	9 678	2,2
Cortex-M0, clang `-Oz`	3 784	8 370	2,2

Tolken före fönstren blir 3,8–4,4 kilobyte i de två bästa, nära annonsens 4. Fönstren och musen mer än fördubblar den. I Z80 tar fönstren 5 884 byte och musen, kön och frågan 3 933; de största nya delarna är ramen, flytten med innehållet och stängningen. Tolken hade knappast fått plats i ett 2764 på 8 kilobyte, utan behövt ett 27128 på 16, som fanns 1984 men kostade mer. Till ett kort med 8 kilobyte hade man kunnat låta ABC80 rita om ett flyttat fönster i stället för att flytta innehållet, rita en enklare ram och stänga med ett kommando i stället för med musen.

Arbetsminnet räcker sämre än förra kapitlet räknade. Fönstertabellen och kön tar 384 byte, och makrona har räknats få resten av de 2 kilobyte bredvid bilden, 1 664 byte. Men ett kort behöver också processorns stack, omkring 250 byte med makron i åtta nivåer, fyllningens stack och en tabell över makrona. Med en stack för fyllningen på 128 poster om tre byte, som räcker för allt utom elaka figurer, hade makrona fått omkring 670 byte, och före fönstren omkring 1 050. Emulatorns tolk är inte ändrad efter det; den visar ett kort med 16 kilobyte EPROM och lite mer arbetsminne än annonsens 18 kilobyte, ett kort som kunde ha byggts men inte HR-80.

8.10 Begränsningarna

– *Påhittat.* Fönstren, musen, `ESC ?` och svaren finns inte i annonsen. Koderna är valda så att de liknar FONSTER, inte efter något riktigt kort.
– *UFD-DOS.* `hr80in` ligger på `$FFBB`–`$FFFA`, där UFD-DOS:en har sin korttabell (`$FFC0`–`$FFE0`) och `UFDUN` (`$FFF9`). FONSTER2 går alltså inte med UFD-DOS, bara utan DOS eller med ABC-DOS, som exempeldisketten har och som lämnar `$FFB0`–`$FFFF` orört.
– *Inga tangenter i FONSTER2.* Slingan frågar kortet och läser inte tangentbordet; programmet stoppas med CTRL-C.
– *Fyllningen* i emulatorns tolk har kvar sin stora stack, som i förra kapitlet.
– *FONSTER* finns inte för BASIC II, där `IF` ska stå sist på raden och tolken hade blivit ett annat program.

8.11 Slutsatserna

Koder i texten, som till en skrivare, räcker för att styra fönster, både i BASIC och på ett kort. Programmet som skriver behöver inte veta var fönstren ligger, och samma tolk med ett tillstånd per tecken gör jobbet i BASIC, i JSP-diagrammet och i kortets C. I BASIC kostar det tid: en slinga per tecken och omritningar nerifrån och upp gör fönstren långsamma, och heltalen mer än halverar tiden.

Med kortet flyttas arbetet från ABC80. Det svåra är inte fönstren, utan vägen tillbaka: ABC80 kan bara ta emot på V24 när det väntar på det, och därför måste kortet vänta på frågan. Inget sparas

under fönstren, varken i BASIC eller på kortet, och ABC80 ritar om det som skymts när kortet säger till. Fönstren och musen hade gjort kortet dyrare, med fyra gånger så mycket EPROM, och de lämnar mindre arbetsminne till makrona.

8.12 Inte prövat

Inget av det här är prövat på en riktig ABC80 eller med ett riktigt kort. Läsrutinens tider är räknade och prövade mot emulatorns sändare, inte mot en riktig.

I kodförrådet ligger programmen i `emulator/exempel/program/`: exemplen `text/FONSTER.BAS` och `text/FONSTER2.BAS`, läsrutinen `hr80in.asm` och musen till provet `fonster2.mus`. Fönstren och musen finns i tolken, `emulator/karna/hr80.c`. Planen, JSP-diagrammet, vad som är ändrat och storleken står i `utanfor/fonster/`, i `PLAN.md`, `README.md`, `LASMIG.md` och `STORLEK.md`.

Forth på ABC80

Kapitlen En snabbare Forth, Forthaid och Forthaid II förutsätter att läsaren kan lite Forth. Band 2 beskriver hur tolken från exempel 7 är byggd, men inte hur den används. Den här bilagan går igenom det som behövs för att följa kapitlen och för att skriva egna kommandon till Forthaid. Allt går att göra på emulatorns webbsida. Välj *Forthaid* bland kretsarna, eller öppna sidan med `?krets=forthaid` efter adressen, och skriv

```
Z=CALL(16384)
```

Tolken svarar `OK` efter varje rad som gick bra, direkt efter det som raden skrev ut. `BYE` går tillbaka till BASIC. Skärmarna nedan är webbsidans, och de ser likadana ut med Forthaid II (`?krets=forthaid2`). Forth-kretsen från band 2 (`?krets=forth`) har samma tolk, men saknar `*`, `U/MOD`, `+!` och orden för BASIC-raderna. Där flyttas först `BOFA` upp med `POKE 65052,0,208` och `NEW`, och tolken startas med `Z=CALL(16384,3)`.

A.1 Stacken

Ett tal som skrivs läggs på stacken. Ett ord tar sina värden från stacken och lämnar sitt resultat där. Ordet `.` tar det senast lagda värdet och skriver ut det. `2 3 + .` lägger alltså 2 och 3 på stacken, `+` ersätter dem med 5, och `.` skriver 5. Det som lades sist kommer först tillbaka, och därför skriver `1 2 3 . . .` talen baklänges:

```
Z=CALL(16384)
2 3 + . 5 OK
1 2 3 . . . 3 2 1 OK
5 DUP . . 5 5 OK
1 2 SWAP . . 1 2 OK
1 2 OVER . . . 1 2 1 OK
1 2 3 ROT . . . 1 3 2 OK
1 2 DROP . 1 OK
6 7 * 4 - . 38 OK
100 7 U/MOD . . 14 2 OK
1 . . 1 23205 STACK ?
 OK
```

Vad ett ord gör med stacken skrivs som en stackkommentar: värdena före och efter, med det senast lagda till höger. `SWAP` är `( a b -- b a )`. Kapitlen och Forthaids manual använder dem, men tolken på ABC80 känner inte till parenteserna, så de skrivs inte in.

Ordet	Stacken	Vad
DUP	(a -- a a)	en kopia
DROP	(a --)	kastar
SWAP	(a b -- b a)	byter plats
OVER	(a b -- a b a)	en kopia av det näst översta
ROT	(a b c -- b c a)	det tredje upp
+ - *	(a b -- c)	a + b, a − b, a · b
U/MOD	(a b -- r k)	kvoten k och resten r av a / b, utan tecken
1+ 1-	(a -- b)	ett mer, ett mindre
= < 0= 0<	(a b -- f)	jämförelserna, sant −1 och falskt 0
.	(a --)	skriver talet

Talen är 16 bitar, från −32 768 till 32 767. Additionen går runt utan fel, så `32767 1+ .` ger −32 768, och `U.` skriver samma bitar utan tecken: `40000 U.` ger 40000, men `40000 .` ger −25 536. Tal skrivs decimalt eller hexadecimalt med `$` framför, som på ABC80 syns som ¤.

Den sista raden på skärmen tar ett värde för mycket. Det andra `.` skriver det som ligger under stacken, kanariefågeln `$5AA5` (23 205), och efter raden ser tolken att stacken är tömd förbi sin botten. Den skriver `STACK ?` och tömmer stackarna. Band 2 beskriver prövningen i avsnittet Stacken kontrolleras, och kapitlet En snabbare Forth vad den kostar.

A.2 Egna ord

Kolon börjar en definition och semikolon avslutar den. Orden däremellan körs inte utan kompileras, och det nya ordet används sedan som de inbyggda, också i nästa definition. En definition kan gå över flera rader, men varje rad är högst 79 tecken, och ett fel tar bort den halvfärdiga definitionen.

```
: KV DUP * ; OK
7 KV . 49 OK
: KUB DUP KV * ; OK
3 KUB . 27 OK
: TECKEN 0< IF ." MINUS "
ELSE ." PLUS " THEN ; OK
-5 TECKEN 5 TECKEN MINUS PLUS OK
: RAKNA 1+ 1 DO I . LOOP ; OK
5 RAKNA 1 2 3 4 5 OK
: NED BEGIN DUP WHILE
DUP . 1- REPEAT DROP ; OK
5 NED 5 4 3 2 1 OK
: HALV BEGIN 2 U/MOD SWAP DROP
DUP . DUP 0= UNTIL DROP ; OK
100 HALV 50 25 12 6 3 1 0 OK
VARIABLE V OK
42 V ! V É . 42 OK
3 V +! V É . 45 OK
25 CONSTANT N OK
N N + . 50 OK
```

`IF` tar ett värde från stacken och kör delen fram till `ELSE` om det inte är 0, annars delen efter. `THEN` avslutar. `."` skriver texten fram till nästa citattecken. Mellanslaget efter `."` hör inte till texten, men det före citattecknet gör det.

`DO` tar gränsen och början, `( g b -- )`, och kör slingan med `I` från b till g − 1. Därför lägger `RAKNA` först till 1 till gränsen. `BEGIN` ... `WHILE` ... `REPEAT` prövar i mitten och slutar när `WHILE` får 0.

`BEGIN` ... `UNTIL` prövar sist och slutar när `UNTIL` får något annat än 0. Styrorden fungerar bara i en definition. Utanför ger de inget fel, men inte heller det väntade.

`VARIABLE V` gör ett ord som lämnar adressen till en plats i minnet. `! ( n a -- )` skriver där, `@ ( a -- n )` läser och `+!` lägger till. ABC80 visar `@` som É, eftersom de har samma kod, och ordet skrivs med tangenten É. `CONSTANT` gör ett ord som lämnar sitt värde. `C@` och `C!` läser och skriver en byte.

Tolken är liten. Kommentarerna med `(` och `\`, `2DUP` och `>` finns inte, och i stället för `a b >` skrivs `b a <` eller `SWAP <`. Det finns inte heller `+LOOP`, `LEAVE`, `DOES>` eller `FORGET` (band 2, avsnittet Det som saknas). `WORDS` listar alla ord, de senast definierade först.

A.3 BASIC-raderna

Forthaids kommandon är Forth-ord, och kretsen har ord för att gå igenom BASIC-programmet. Med dem kan egna kommandon skrivas. En rad i programmet börjar med en byte med radens längd och två byte med radnumret, och efter den sista raden står en byte 1. `BOFA` är adressen `$FE1C`, där adressen till programmets början står (band 1, kapitlet Systemvariabler och externa adresser).

Ordet	Stacken	Vad
`BOFA EOFA HEAP`	`( -- a )`	adressen till BASIC:s pekare, läses med `@`
`FINDLN`	`( n -- r )`	den första raden med numret n eller högre
`LINE#`	`( r -- n )`	radens nummer
`NEXTLINE`	`( r -- r' )`	nästa rad
`END-PRG?`	`( r -- f )`	sant vid programmets slut
`LISTLINE`	`( r -- n )`	listar raden, n är skärmraderna den tog
`H.`	`( u -- )`	talet hexadecimalt, fyra siffror

Här har ett program på fyra rader skrivits in först, och sedan startas tolken. `RADER` räknar raderna, och `LANGA` listar de rader som är längre än ett antal byte:

```
Z=CALL(16384)
BOFA É H. C000 OK
BOFA É LINE# . 10 OK
BOFA É NEXTLINE LINE# . 20 OK
BOFA É C@ . 28 OK
: RADER  0 BOFA É
BEGIN DUP END-PRG? 0= WHILE
NEXTLINE SWAP 1+ SWAP REPEAT
DROP . ; OK
RADER 4 OK
: LANGA  BOFA É
BEGIN DUP END-PRG? 0= WHILE
OVER OVER C@ < IF
DUP LISTLINE DROP THEN
NEXTLINE REPEAT DROP DROP ; OK
15 LANGA 10 FOR I=1 TO 3
20 PRINT "VARV";I
OK
BYE
ABC80
```

`RADER` har räknaren under radens adress på stacken, `( antal r )`, och byter plats på dem när räknaren ska ökas. `LANGA` har gränsen under radens adress, `( g r )`. `OVER OVER` gör en kopia

av båda, `C@` läser radens längd, och `<` jämför gränsen med den. Rad 10 är 28 byte och rad 20 är 21. `NEXT I` och den sista raden är 14 byte och listas inte. Längden gäller raden som den ligger kompilerad i minnet, och band 1 beskriver den formen i kapitlet Satserna kompileras.

Forthaids egna kommandon är skrivna på samma sätt. `FIND` och `VAR` gick först igenom raderna så här, med slingor i Forth, och fick sedan sina tyngsta delar i maskinkod (kapitlet Forthaid, avsnittet Tiderna). De egna orden får 304 byte i Forthaid och 272 i Forthaid II. `Z=CALL(16384)` glömmer dem, och `Z=CALL(16384,1)` startar tolken med dem kvar. I Forthaid II sparar `SAVEW` dem på skivan och `LOADW` läser tillbaka dem, och på exempeldisketten ligger `VERKTYG.FTH` med fyra färdiga kommandon (kapitlet Forthaid II).

COMAL 0,5 på ABC80

Kapitlet COMAL 0,5 beskriver hur strukturerna är byggda och vad de kostar. Den här bilagan visar hur de används, med ett program som har alla slagen. Välj *COMAL 0,5* bland kretsarna på emulatorns webbsida, eller öppna den med `?krets=comal`. Maskinen startar om med den gamla ROM:en (11273), de 23 ändrade byten och kretsen på `$4000`. Allt annat är ABC80-BASIC som förut, och gamla program går som de gjorde.

B.1 Det nya

Satserna står på egna rader, och varje block stängs med sin egen sats: `WHILE` med `ENDWHILE`, `REPEAT` med `UNTIL`, `IF` med `ENDIF`, `CASE` med `ENDCASE` och `PROC` med `ENDPROC`. Tabellen i kapitlet COMAL 0,5, avsnittet Satserna, har alla satserna och deras fel. Några regler skiljer sig från andra språk med samma satser:

- `IF` med ingenting efter `THEN` öppnar ett block. Står det en sats efter `THEN` är det ett vanligt `IF` på en rad, som förut.
- Typen ska stämma i anropet. Ett heltal skrivs med `%`, också en konstant: `EXEC P(5%)`, inte `EXEC P(5)`. Fel typ eller fel antal ger fel 72.
- Parametrarna tar värdet, inte variabeln. `EXEC` sparar parametrarnas och de lokala variablernas gamla värden, och `ENDPROC` lägger tillbaka dem, så att de är som förut efter anropet. Rekursion fungerar. Strängar kan inte vara parametrar.
- `LOCAL` står direkt efter `PROC`, annars fel 72.
- `WHEN` tar ett eller flera värden med komma emellan. Stämmer ingen och det inte finns någon `OTHERWISE` blir det fel 73.
- Blocken paras vid `RUN`, som `FOR` och `NEXT`. Ett block som inte stängs ger fel 70 innan något körs, och en stängning utan början eller av fel slag fel 71.
- `GOTO` får hoppa ut ur `WHILE`, `REPEAT`, `IF` och `CASE`. Blocken håller sina platser i koden och inte på stacken, så inget blir kvar.
- Variabelnamnen är ABC80-BASIC:ens, en bokstav och en siffra. Långa namn, `FUNC` och `REF` finns inte.

B.2 Ett program

Programmet räknar summan 1 + 2 + ... + n för n från 1 till 6 och säger hur stor den är. Raderna skrivs in utan indrag, och `LIST` drar in dem två steg per nivå:

```
10 N%=1%
20 WHILE N%<=6%
30   EXEC VISA(N%)
40   N%=N%+1%
50 ENDWHILE
60 END
100 PROC VISA(K%)
110   LOCAL S%
120   S%=0%
```

```
130   REPEAT
140     S%=S%+K%
150     K%=K%-1%
160   UNTIL K%=0%
170   PRINT S%;" ";
180   CASE S% OF
190   WHEN 1%,3%
200     PRINT "LITET"
210   WHEN 6%,10%
220     PRINT "MELLAN"
230   OTHERWISE
240     IF S%<20% THEN
250       PRINT "STORT"
260     ELSE
270       PRINT "MYCKET STORT"
280     ENDIF
290   ENDCASE
300 ENDPROC
```

WHEN och OTHERWISE står på samma nivå som CASE, och det som hör till dem ett steg in. Efter RUN skrivs K% och S% ut:

```
RUN
 1 LITET
 3 LITET
 6 MELLAN
 10 MELLAN
 15 STORT
 21 MYCKET STORT

ABC80
PRINT K%,S%
 0             0

ABC80
```

VISA räknar ned K% till 0 i sin slinga och använder S%, men båda är 0 efteråt, som innan programmet kördes. Det är parametern och den lokala variabeln som lades tillbaka. Huvudprogrammets N% påverkas inte av att VISA ändrar K%.

Två fel, det ena i anropet och det andra i ett block. Rad 30 skrivs om med en parameter utan %, och sedan tas ENDWHILE bort:

```
30 EXEC VISA(N)
RUN

ERR 72 LINE 30
ABC80
30 EXEC VISA(N%)
50
RUN

ERR 70 LINE 20
ABC80
```

Fel 72 gäller anropet på rad 30. Fel 70 kommer redan när blocken paras vid RUN, och då har ingen rad körts. Raden i felet är WHILE, det block som aldrig stängdes. Ett program som flyttar Hanois torn med en rekursiv procedur står i kapitlet COMAL 0,5, avsnittet Satserna.